

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communication Technology



TECHNICAL REPORT

**USING MACHINE LEARNING TO
DETECT MALICIOUS LINKS**

Students:	Do Thanh Trung	202416833
	Nguyen Tuan Minh	202416811
	Khong Tuan Anh	202416775
	Nguyen Hoang Anh	202416773
	Pham Duc Linh	202416806

Date: June 2026

Contents

I	Introduction	2
1	System Overview	2
2	Anatomy of a Uniform Resource Locator (URL)	2
II	Binary Classification via Static Lexical Analysis	3
3	Training Data Sources	3
3.1	International Dataset (Kaggle) and Label Aggregation	3
3.2	Vietnam-Specific Dataset (Tin Nhiem Mang)	4
3.3	Global Whitelist (Tranco List)	4
4	Processing Pipeline	4
5	Whitelist and Trusted Domain Resolution	12
5.1	Tier 1 — Static Whitelist	12
5.2	Tier 2 — eTLD-Aware Brand Resolution via tldextract	12
6	Hard Rule Engine	13
6.1	Rule 0 — URL Shortener Detection (Pre-Whitelist)	13
6.2	Rule 1 — Brand Spoofing (Normal TLD)	14
6.3	Rule 2 — Brand Spoofing (Suspicious TLD)	14
6.4	Rule 3 — C2 Server Callback Detection (Random Subdomain)	14
6.5	Rule 4 — C2 Server Callback Detection (High-Entropy Subdomain)	14
6.6	Rule 5 — Executable Distribution via Random Subdomain	14
6.7	Rule 6 — Executable Distribution via Free Hosting	15
6.8	Rule 7 — Malware Distribution via Free Hosting	15
7	Feature Engineering	15
7.1	Structural & Length Features	23
7.2	Statistical & Entropy Features	24
7.3	Lexical & Heuristic Features	25
7.4	Advanced Cyber Threat Indicators	26
7.5	Executable File Detection Features	27
7.6	Feature Summary	28
8	Model Training	28
8.1	Data Loading and Label Aggregation	31
8.2	Random Forest Hyperparameters	32
8.3	XGBoost Hyperparameters	32
8.4	LightGBM Hyperparameters	32

8.5	Ensemble Weight Search via F_2 Score	33
9	Ensemble Learning Model	33
9.1	Random Forest (RF)	34
9.2	XGBoost	34
9.3	LightGBM	34
9.4	Logistic Regression (Baseline)	35
9.5	Weighted Average Ensemble Strategy	35
9.6	Method Comparison	36
10	Dynamic Thresholding Mechanism	36
10.1	Context Signals	37
10.2	Threshold Selection Rules	38
10.3	Vietnamese Domain Protection (<code>clean_vn</code>)	39
10.4	Clean Generic Domain Protection (<code>clean_domain</code>)	40
11	Limitations and Future Work	40
11.1	Lexical-Only Analysis	40
11.2	Training Data Staleness	40
11.3	Sensitivity to URL Normalization	41
11.4	Over-Reliance on Hard Rules	41
11.5	Geographic and Linguistic Bias	41
11.6	Ensemble Collapse	41
11.7	Dynamic Threshold Fragility	42
11.8	Whitelist Maintenance Burden	42
11.9	Future Work	42
12	Conclusion	43
III	Multi-class Classification via Static Lexical Analysis	44
13	Training Data Sources	44
13.1	Primary Dataset: Kaggle Malicious Phish Corpus	44
13.2	Supplementary Dataset: URLhaus Malware Feed	44
13.3	Whitelist Reference: Trusted Domain Catalogue	44
14	Processing Pipeline	45
14.1	URL Normalisation	46
14.2	Inference Pipeline	47
15	Feature Engineering	48
15.1	Structural and Presence Features	48
15.2	Length Features	49
15.3	Statistical and Entropy Features	50
15.4	Special Character Features	51

15.5	Lexical and Heuristic Features	52
15.5.1	Levenshtein Similarity with the Whitelist	54
15.5.2	Suspicious Keyword Taxonomy	54
15.6	Advanced Cyber Threat Indicators	55
15.7	Feature Summary	57
16	Model Training	57
16.1	Data Preprocessing	57
16.1.1	Label Encoding	57
16.1.2	Feature Matrix Construction	57
16.1.3	Deduplication	57
16.2	Data Partitioning via Stratified Group <i>K</i> -Fold	58
16.3	LightGBM Hyperparameter Configuration	59
16.4	Training Procedure	60
16.5	Evaluation Results	60
17	Model Architecture	61
17.1	Gradient Boosted Decision Trees	61
17.2	Objective Function	61
17.3	Class Balancing	61
17.4	Multi-Layer Regularisation Strategy	62
18	Conclusion	62
18.1	Limitations and Future Work	63

Part I

Introduction

Abstract

This technical report presents a comprehensive malicious URL classification system that relies exclusively on static lexical analysis of Uniform Resource Locator (URL) strings. The system operates entirely without issuing network requests, DNS queries, or page content retrieval, enabling zero network latency during inference. The project is divided into two primary pipelines: (1) Binary Classification using an Ensemble of Machine Learning models to differentiate between Safe and Dangerous URLs, and (2) Multi-class Classification using a LightGBM Gradient Boosted Decision Tree (GBDT) model to categorize URLs into benign, defacement, malware, and phishing. Extensive feature engineering is employed to extract structural, statistical, lexical, and advanced cyber-threat indicators. Both approaches demonstrate high accuracy, sub-millisecond latency per URL, and robust resilience against evasive techniques, making the solution highly suitable for real-time cybersecurity integrations.

1 System Overview

Detecting malicious websites is a classic yet ever-evolving challenge in the field of Information Security. Traditional methods, such as utilizing blacklists, suffer from high latency and the inability to detect Zero-day attacks. This report presents a comprehensive malicious URL classification system that relies exclusively on static lexical analysis of Uniform Resource Locator (URL) strings. By avoiding network requests or page content retrieval, the system operates with ultra-fast response times, making it highly suitable for real-time integration. The project is divided into two main approaches: Binary Classification (detecting Safe vs. Dangerous URLs) using an Ensemble of Machine Learning models, and Multi-class Classification (categorizing into Benign, Defacement, Malware, and Phishing) using LightGBM Gradient Boosting.

2 Anatomy of a Uniform Resource Locator (URL)

A Uniform Resource Locator (URL) is a reference to a web resource that specifies its location on a computer network and a mechanism for retrieving it. Structurally, a URL can be decomposed into several distinct components:

- **Scheme (Protocol):** Indicates the protocol used to access the resource (e.g., `http`, `https`, `ftp`).

- **Network Location (Netloc):** Contains the authentication credentials (username/password), the host (domain name or IP address), and an optional port number.
- **Domain & Subdomain:** The host is often split into a registered domain (e.g., `example.com`) and subdomains (e.g., `www`, `api`). The Top-Level Domain (TLD) is the suffix (e.g., `.com`, `.vn`).
- **Path:** Specifies the specific resource or directory structure on the server.
- **Query Parameters:** Follows a question mark (?) and contains data to be passed to the server, typically as key-value pairs separated by ampersands (&).
- **Fragment:** Follows a hash (#) and points to a specific section within the retrieved document.

By analyzing the lexical features, character distribution, and structural anomalies across these components, the proposed system can effectively identify malicious patterns without rendering the actual web page.

Part II

Binary Classification via Static Lexical Analysis

3 Training Data Sources

The success of a Machine Learning model heavily depends on the quality and coverage of its training dataset. To ensure the system can identify a wide range of global attack vectors while remaining optimized for the Vietnamese cyberspace, the dataset was aggregated and pre-processed from three primary sources:

3.1 International Dataset (Kaggle) and Label Aggregation

The core data was extracted from the public “**Malicious URLs dataset**” on Kaggle (authored by *sid321axn*). Originally, this dataset provides hundreds of thousands of URL samples structured as a multi-class classification problem with four distinct morphological labels:

- **Benign:** Safe and legitimate websites.
- **Phishing:** Fraudulent websites designed to steal credentials.
- **Malware:** Links used to distribute malicious software.

- **Defacement:** Legitimate websites that have been compromised and altered by hackers.

However, to align with the core objective of the project—which is to provide a decisive, real-time warning to end-users (Safe vs. Dangerous)—we implemented a **Binary Label Aggregation** strategy. All malicious sub-categories (*Phishing*, *Malware*, *Defacement*) were merged into a single positive class labeled as 1 (Dangerous), while *Benign* remained as class 0 (Safe).

This approach not only simplifies the decision boundary for Tree-based models (XG-Boost and Random Forest) but also significantly reduces noise and class imbalance issues during the training phase, thereby maximizing the model’s recall rate for any type of cyber threat.

3.2 Vietnam-Specific Dataset (Tin Nhiem Mang)

A common drawback of international datasets is the omission of targeted phishing campaigns aimed specifically at Vietnamese users. To address this, the development team performed automated data collection from the **Tin Nhiem Mang** project (tinnhiemmang.vn), operated by the National Cyber Security Center (NCSC) of Vietnam.

By integrating this localized dataset, the system learned to recognize the structural patterns of legitimate Vietnamese entities (`.gov.vn`, `.edu.vn`) while effectively identifying spoofed domestic brands (e.g., local banks and e-commerce platforms).

3.3 Global Whitelist (Tranco List)

Furthermore, to ensure the model correctly identifies a wide spectrum of legitimate international websites and avoids bias (False Positives), we constructed an extensive **Global Whitelist** using data from the **Tranco List**.

The Tranco dataset provides a highly reliable, research-grade ranking of the top millions of websites globally. By incorporating a subset of these top-ranking domains into our training pipeline, the model achieved a robust understanding of benign URL structures at an international scale, effectively balancing the aggressive detection of the heuristic rules.

4 Processing Pipeline

The system processes an input URL through five sequential defense layers:

1. **Shortener Pre-Check:** Detects URL shortening services before any whitelist bypass.
2. **Whitelist Check:** Bypasses further inspection for known reputable domains.

3. **Hard Rules:** Handles explicitly malicious behaviors with deterministic verdicts.
4. **ML Ensemble:** Analyzes URL features using XGBoost, LightGBM, Random Forest, & Logistic Regression.
5. **Dynamic Thresholding:** Adjusts sensitivity based on camouflage techniques and domain context.

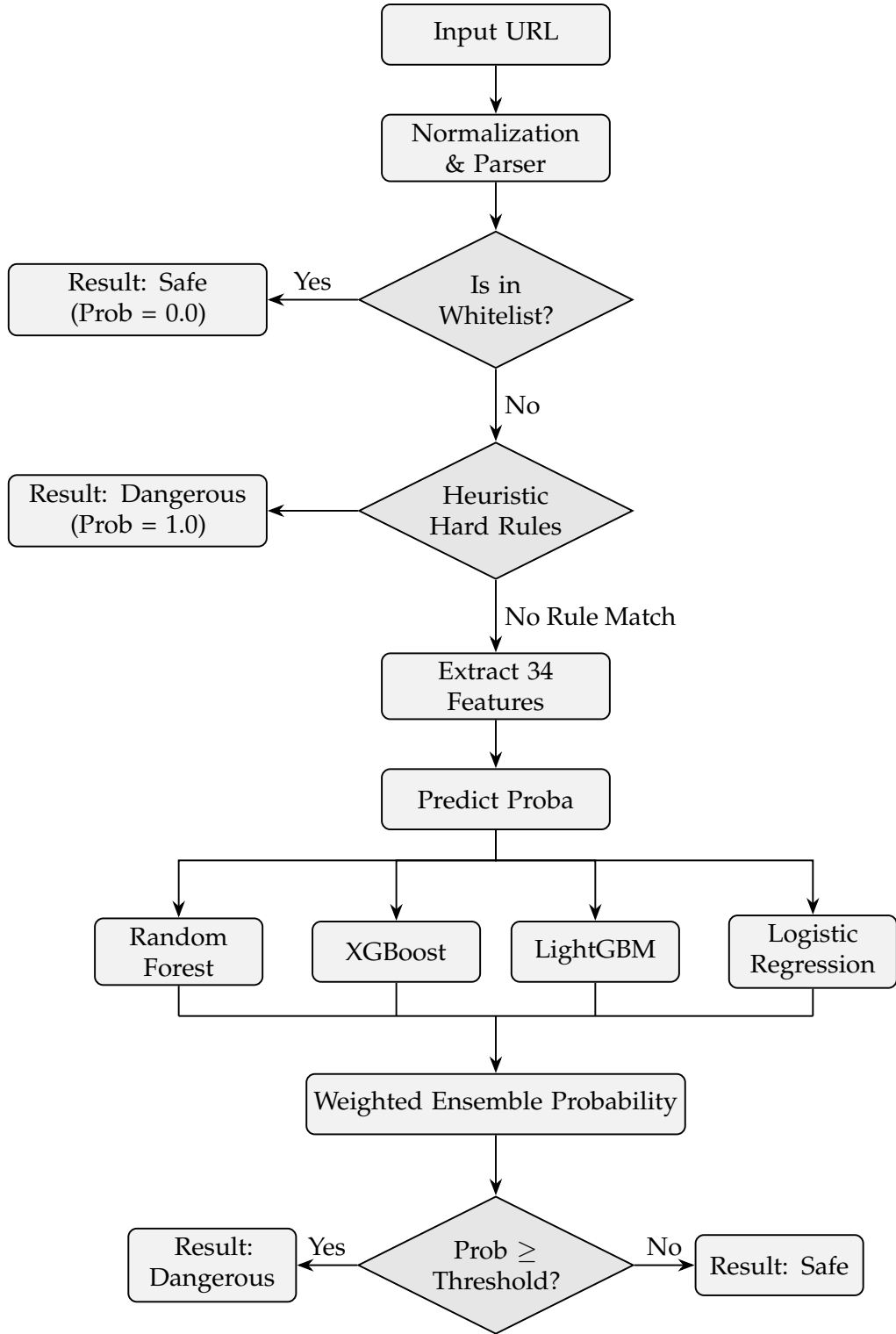


Figure 1: Pipeline Architecture of the Detection System

The entry point is the `predict_single_url()` function in `predict.py`. After normalizing the input (prepending `https://` if no scheme is present and stripping trailing slashes and `www.`), it delegates to the five layers in order. Below is the complete implementation:

Listing 1: predict.py — Full prediction pipeline (RF + XGB ensemble with hard rules)

```

1 import re
2 import json
3 from urllib.parse import urlparse
4
5 import pandas as pd
6 import xgboost as xgb
7 import joblib
8
9 from src.feature import extracting_features, is_trusted_domain,
    SHORTENERS
10
11 # Load models and weights
12 rf_model = joblib.load('models/rf_model.pkl')
13 xgb_model = xgb.XGBClassifier()
14 xgb_model.load_model('models/xgb_model.json')
15 lgb_model = joblib.load('models/lgb_model.pkl')
16 lr_model = joblib.load('models/lr_model.pkl')
17
18 _w = json.load(open('models/weights.json'))
19 W_RF, W_XGB, W_LGB, W_LR = _w['w_rf'], _w['w_xgb'], _w.get('
    w_lgb', 0.0), _w.get('w_lr', 0.0)
20
21 TRACKING_PARAMS = [
22     'utm_source', 'utm_medium', 'utm_campaign',
23     'fbclid', 'gclid', 'sp_atk', 'skuid',
24     'ref', 'aid', 'label', 'utm_term', 'utm_content', 'utm_id',
25     'wbraid', 'gbraid', 'dclid',
26     'ttclid', 'igshid', 'twclid', 'msclkid', 'li_fat_id', 'yclid',
27     'mc_cid', 'mc_eid', 'mkt_tok', 'hsCtaTracking',
28     'aff_id', 'affid', 'click_id', 'clickid', 'cid', 'subid', '
    tag',
29     'PHPSESSID', 'JSESSIONID', '_ga', '_gl', 's_kwcid', '
    gad_source'
30 ]
31
32 def predict_single_url(url: str) -> dict:
33     url = str(url).strip()
34     if not url.startswith(('http://', 'https://')):
35         url = 'https://' + url
36
37     # Normalize: always strip trailing slashes so "example.com/"
38     == "example.com"

```

```

37 url = url.rstrip('/')
38
39 clean = re.sub(r'^https?://', '', url.lower())
40 clean = re.sub(r'^www\.', '', clean)
41 try:
42     parsed = urlparse('// ' + clean)
43 except Exception:
44     parsed = urlparse('// ')
45 hostname = parsed.netloc.split(':')[0]
46 query = parsed.query or ''
47
48 # Shortener check (must come BEFORE whitelist: bit.ly is in
top-1M
49 # but any shortened URL is inherently suspicious regardless)
50 is_shortener_host = (
51     hostname in SHORTENERS or
52     any(hostname.endswith('.') + s for s in SHORTENERS)
53 )
54 if is_shortener_host:
55     feat = extracting_features(url)
56     X = pd.DataFrame([feat])
57     p_rf = rf_model.predict_proba(X)[0][1]
58     p_xgb = xgb_model.predict_proba(X)[0][1]
59     p_lgb = lgb_model.predict_proba(X)[0][1]
60     p_lr = lr_model.predict_proba(X)[0][1]
61     prob = W_RF * p_rf + W_XGB * p_xgb + W_LGB * p_lgb +
W_LR * p_lr
62     return {
63         'prediction': 1, 'label': 'dangerous', 'prob': round
(prob, 4),
64         'prob_rf': round(p_rf, 4), 'prob_xgb': round(p_xgb,
4),
65         'prob_lgb': round(p_lgb, 4), 'prob_lr': round(p_lr,
4),
66         'source': 'hard_rule', 'reason': f'URL shortener: {
hostname}',
67     }
68
69 # Whitelist check
70 if is_trusted_domain(hostname):
71     return {
72         'prediction': 0, 'label': 'safe', 'prob': 0.0,
73         'source': 'whitelist', 'reason': f'Trusted domain: {
hostname}',
74     }

```

```

75
76     feat = extracting_features(url)
77
78     # Hard rules (priority over ML)
79
80     # Brand spoof
81     if feat['brand_similarity'] >= 0.75 and feat['tld_suspicious'] == 0:
82         return {
83             'prediction': 1, 'label': 'dangerous', 'prob': 1.0,
84             'source': 'hard_rule',
85             'reason': f'Brand similarity: {feat["brand_similarity"]}',
86         }
87     if feat['tld_suspicious'] == 1 and feat['brand_similarity'] >= 0.65:
88         return {
89             'prediction': 1, 'label': 'dangerous', 'prob': 1.0,
90             'source': 'hard_rule',
91             'reason': f'Suspicious TLD + brand similarity: {feat["brand_similarity"]}',
92         }
93
94     # UUID path + subdomain random -> C2/malware callback
95     if feat['has_uuid_path'] == 1 and feat['subdomain_is_random'] == 1:
96         return {
97             'prediction': 1, 'label': 'dangerous', 'prob': 1.0,
98             'source': 'hard_rule',
99             'reason': 'Random subdomain + UUID path/query - likely C2 callback',
100        }
101
102     # UUID path + high subdomain entropy -> C2 callback (subdomain_is_random might miss)
103     if feat['has_uuid_path'] == 1 and feat['subdomain_entropy'] >= 2.4:
104         return {
105             'prediction': 1, 'label': 'dangerous', 'prob': 1.0,
106             'source': 'hard_rule',
107             'reason': f'UUID path + high-entropy subdomain ({feat["subdomain_entropy"]}) - likely C2 callback',
108        }
109
110     # Executable file + random subdomain -> malware served from

```

```

111 random VPS
112     if feat['has_executable_ext'] == 1 and feat['
113 subdomain_is_random'] == 1:
114         return {
115             'prediction': 1, 'label': 'dangerous', 'prob': 1.0,
116             'source': 'hard_rule',
117             'reason': 'Executable file extension + random
118 subdomain - likely malware distribution',
119         }
120
121     # Executable file + free hosting -> malware distribution
122     if feat['has_executable_ext'] == 1 and feat['is_free_hosting
123 '] == 1:
124         return {
125             'prediction': 1, 'label': 'dangerous', 'prob': 1.0,
126             'source': 'hard_rule',
127             'reason': 'Executable file extension + free hosting
128 - likely malware distribution',
129         }
130
131     # Free hosting + download -> malware distribution
132     if feat['free_hosting_download'] == 1:
133         return {
134             'prediction': 1, 'label': 'dangerous', 'prob': 1.0,
135             'source': 'hard_rule',
136             'reason': 'Free hosting + download param - likely
137 malware distribution',
138         }
139
140     # ML prediction
141     X = pd.DataFrame([feat])
142     p_rf = rf_model.predict_proba(X)[0][1]
143     p_xgb = xgb_model.predict_proba(X)[0][1]
144     p_lgb = lgb_model.predict_proba(X)[0][1]
145     p_lr = lr_model.predict_proba(X)[0][1]
146     prob = W_RF * p_rf + W_XGB * p_xgb + W_LGB * p_lgb + W_LR *
147 p_lr
148
149     _tld = feat.get('tld_suspicious', 1)
150     _vn_etlds = {'vn', 'com.vn', 'net.vn', 'org.vn', 'edu.vn', '
151 gov.vn',
152                 'int.vn', 'ac.vn', 'biz.vn', 'info.vn', 'name.
153 vn', 'pro.vn'}
154     from tldextract import extract as _tldext
155     _ext = _tldext(hostname)

```

```

147     _suffix = _ext.suffix.lower() if _ext.suffix else ''
148     is_vn_domain = _suffix in _vn_etlds
149
150     clean_vn = is_vn_domain and (
151         feat['brand_similarity'] < 0.75 and
152         feat['has_suspicious_word'] == 0 and
153         feat['is_ip'] == 0 and
154         feat['subdomain_is_random'] == 0 and
155         feat['is_free_hosting'] == 0 and
156         feat['has_executable_ext'] == 0
157     )
158
159     clean_domain = (
160         feat['tld_suspicious'] == 0 and
161         feat['brand_similarity'] < 0.65 and
162         feat['has_suspicious_word'] == 0 and
163         feat['is_ip'] == 0 and
164         feat['subdomain_is_random'] == 0 and
165         feat['is_free_hosting'] == 0 and
166         feat['has_executable_ext'] == 0
167     )
168
169     # Threshold determining
170
171     q_count = len(query.split('&')) if query else 0
172     has_tracking = any(p in query.lower() for p in
TRACKING_PARAMS)
173     is_short_sus = feat['is_shortener'] == 1 or (
174         feat['url_len'] < 30 and feat['
path_digit_ratio'] > 0.3)
175     is_word_salad = feat['url_len'] > 60 and feat['
has_suspicious_word'] == 1
176     url_long_clean = (
177         feat['url_len'] > 80 and q_count > 3 and has_tracking
178         and feat['tld_suspicious'] == 0 and feat['is_ip'] == 0
179     )
180
181     if is_short_sus:
182         threshold = 0.30
183     elif is_word_salad:
184         threshold = 0.35
185     elif url_long_clean:
186         threshold = 0.60
187     elif clean_vn:
188         threshold = 0.92

```

```

189     elif clean_domain:
190         threshold = 0.85
191     else:
192         threshold = 0.45
193
194
195
196     prediction = 1 if prob >= threshold else 0
197
198     return {
199         'prediction': prediction,
200         'label':      'dangerous' if prediction == 1 else 'safe'
201     },
202     {
203         'prob':      round(prob, 4),
204         'prob_rf':   round(p_rf, 4),
205         'prob_xgb':  round(p_xgb, 4),
206         'prob_lgb':  round(p_lgb, 4),
207         'prob_lr':   round(p_lr, 4),
208         'source':    'model',
209         'threshold': threshold,
210     }

```

5 Whitelist and Trusted Domain Resolution

The first layer of the pipeline performs a whitelist check before any feature extraction takes place (with the exception of URL shortener detection, which runs even earlier). The `is_trusted_domain()` function in `feature.py` implements a two-tier trust resolution strategy.

5.1 Tier 1 — Static Whitelist

The system maintains a hardcoded set of 27 high-confidence trusted domains (e.g., `google.com`, `facebook.com`, `vietcombank.com.vn`) supplemented by a dynamic whitelist loaded at startup from `data/whitelist.txt` (capped at 5,000 entries, generated from the Tranco Top-1M list). A hostname passes this tier if it exactly matches any whitelisted domain or is a subdomain thereof (suffix check).

5.2 Tier 2 — eTLD-Aware Brand Resolution via `tldextract`

A critical edge case arises with country-code second-level TLDs (ccSLDs) such as `.co.uk`, `.com.vn`, and `.com.au`. A naïve suffix extraction would misidentify the SLD of `microsoft.co.uk` as `co` rather than `microsoft`, causing legitimate brand domains to fail the trust check.

The system handles this by delegating eTLD resolution to the `tlldextract` library, which maintains an up-to-date copy of the Public Suffix List. Given a hostname, `tlldextract.extract()` returns three components: `subdomain`, `domain` (SLD), and `suffix` (effective TLD). The trust check then verifies:

- The extracted domain (SLD) is a member of the `FAMOUS_BRANDS` set.
- The leaf label of the suffix (i.e., the rightmost TLD component) is *not* in the `SUSPICIOUS_TLDS` set.

If both conditions hold, the domain is granted full trust and the pipeline returns `safe` immediately.

Input Hostname	Extracted SLD	TLD	Trusted?
<code>microsoft.co.uk</code>	<code>microsoft</code>	<code>uk</code>	Yes
<code>vietcombank.com.vn</code>	<code>vietcombank</code>	<code>vn</code>	Yes (static whitelist)
<code>paypal.com</code>	<code>paypal</code>	<code>com</code>	No (not in <code>FAMOUS_BRANDS</code>)
<code>google.xyz</code>	<code>google</code>	<code>xyz</code>	No (TLD is suspicious)

Table 1: Examples of eTLD-aware brand resolution outcomes

6 Hard Rule Engine

Before invoking the machine learning ensemble, the pipeline applies a deterministic rule engine that short-circuits evaluation and immediately returns `prediction = 1` (dangerous) with `prob = 1.0` whenever a URL matches a known malicious pattern. These rules encode domain-expert knowledge that does not require probabilistic inference—if the condition is met, the verdict is certain.

There are seven independent rules, evaluated in order. The first match terminates the pipeline.

6.1 Rule 0 — URL Shortener Detection (Pre-Whitelist)

$$\text{hostname} \in \text{SHORTENERS}$$

This rule runs *before* the whitelist check. URL shortening services (e.g., `bit.ly`, `tinyurl.com`, `t.co`) are frequently abused to obscure malicious destinations. Even though some shorteners may appear in the Tranco Top-1M whitelist, any shortened URL is inherently suspicious because the final destination is hidden from the user. When triggered, the ML ensemble still runs to produce a probability score for informational purposes, but the prediction is forced to 1 (dangerous) with the source marked as `hard_rule`.

6.2 Rule 1 — Brand Spoofing (Normal TLD)

$$\text{brand_similarity} \geq 0.75 \quad \text{AND} \quad \text{tld_suspicious} = 0$$

A domain whose Second-Level Domain (SLD) is highly similar to a known brand name (similarity score ≥ 0.75) *and* which uses a legitimate, non-flagged TLD is almost certainly a spoofed site. The use of a reputable TLD (e.g., .com, .net) is itself a social engineering tactic designed to lower the victim's guard. Example: paypal.com, google.net.

6.3 Rule 2 — Brand Spoofing (Suspicious TLD)

$$\text{tld_suspicious} = 1 \quad \text{AND} \quad \text{brand_similarity} \geq 0.65$$

When the TLD already belongs to the suspicious set (e.g., .xyz, .tk, .top), the brand similarity threshold is relaxed from 0.75 down to 0.65. The rationale is that a suspicious TLD is itself a strong independent signal; even a moderate brand resemblance in this context is highly indicative of a phishing campaign. Example: facebook.xyz, shopee-login.top.

6.4 Rule 3 — C2 Server Callback Detection (Random Subdomain)

$$\text{has_uuid_path} = 1 \quad \text{AND} \quad \text{subdomain_is_random} = 1$$

Command-and-Control (C2) infrastructure used by malware often follows a characteristic pattern: a randomly generated subdomain (e.g., euftrhnx) paired with a UUID-formatted path segment (e.g., /ba1019ee-a048-4bd5-a90d-1fc5da2b8696). The subdomain provides Domain Generation Algorithm (DGA) evasion, while the UUID serves as a unique victim or session identifier. Flagging both signals simultaneously yields a near-zero false positive rate for this class of threat.

6.5 Rule 4 — C2 Server Callback Detection (High-Entropy Subdomain)

$$\text{has_uuid_path} = 1 \quad \text{AND} \quad \text{subdomain_entropy} \geq 2.4$$

This rule is a fallback for Rule 3 that catches C2 patterns where the subdomain_is_random heuristic may not fire (e.g., subdomains with some vowels but still machine-generated). A UUID in the path combined with high Shannon entropy in the subdomain remains a strong indicator of automated C2 infrastructure.

6.6 Rule 5 — Executable Distribution via Random Subdomain

$$\text{has_executable_ext} = 1 \quad \text{AND} \quad \text{subdomain_is_random} = 1$$

URLs that serve executable file types (.exe, .msi, .apk, .sh, etc.) from domains with machine-generated subdomains are strong indicators of malware distribution from

compromised or ephemeral VPS infrastructure.

6.7 Rule 6 — Executable Distribution via Free Hosting

`has_executable_ext = 1 AND is_free_hosting = 1`

Legitimate software vendors do not distribute executables from free developer hosting platforms (e.g., `vercel.app`, `netlify.app`, `github.io`). This combination is almost exclusively associated with malware dropper campaigns.

6.8 Rule 7 — Malware Distribution via Free Hosting

`free_hosting_download = 1`

This is a composite feature (see Section 15) that fires when a URL is hosted on a free platform *and* the path or query string contains a download or installation keyword. This pattern is strongly associated with malware dropper campaigns.

Rule	Condition	Threat Class
0	Hostname in SHORTENERS	Obfuscated destination (pre-whitelist)
1	$\text{brand_sim} \geq 0.75$ + non-suspicious TLD	Phishing (brand spoof, trusted-looking domain)
2	$\text{tld_suspicious} + \text{brand_sim} \geq 0.65$	Phishing (brand spoof, low-quality domain)
3	UUID path + random subdomain	Malware C2 callback / DGA
4	UUID path + subdomain entropy ≥ 2.4	Malware C2 callback (fallback)
5	Executable extension + random subdomain	Malware distribution (random VPS)
6	Executable extension + free hosting	Malware distribution (free platform)
7	Free hosting + download/install in path	Malware dropper distribution

Table 2: Summary of hard rule conditions and corresponding threat classes

7 Feature Engineering

The system extracts 34 numerical features from each URL using purely lexical analysis—no DNS resolution, no HTTP request, no HTML parsing. All features are computed solely from the URL string by the `extracting_features()` function in `feature.py`. This ensures sub-millisecond feature extraction and eliminates any external dependency. The full implementation is listed below:

Listing 2: `feature.py` — Feature extraction and domain trust resolution

```

1 from urllib.parse import urlparse
2 import re
3 import math
4 from rapidfuzz import fuzz
5 import tldextract
6
7 SUSPICIOUS_WORDS = ['login', 'secure', 'account', 'update', '
    verify', 'bank',
8                       'paypal', 'signin', 'confirm', 'password', '
    free', 'lucky',
9                       'click', 'prize', 'bonus']
10
11 SUSPICIOUS_TLDS = {
12     'tk', 'ml', 'ga', 'cf', 'gq', 'xyz', 'top', 'click', 'link',
13     'win', 'download', 'racing', 'vip', 'club', 'online', 'site'
14     ,
15     'live', 'fun', 'space', 'pw', 'christmas', 'cyou', 'rest',
16     'icu', 'cfd', 'hair', 'makeup', 'monster', 'quest', 'skin',
17     'football', 'cricket', 'golf', 'soccer', 'rugby', '
    basketball',
18     'bet', 'casino', 'poker', 'bingo', 'lottery',
19 }
20
21 TRUSTED_DOMAINS = {
22     'google.com', 'googleapis.com', 'googleusercontent.com',
23     'facebook.com', 'instagram.com', 'youtube.com',
24     'microsoft.com', 'office.com', 'live.com',
25     'amazon.com', 'amazonaws.com', 'github.com', '
    githubusercontent.com',
26     'cloudflare.com', 'akamai.com', 'shopee.vn', 'tiki.vn', '
    lazada.vn',
27     'vietcombank.com.vn', 'bidv.com.vn', 'vnexpress.net', '
    tuoitre.vn',
28     'dantri.com.vn', 'gov.vn', 'edu.vn', 'zalo.me', 'momo.vn', '
    zalopay.vn', 'vlr.gg'
29 }
30
31 FAMOUS_BRANDS = {
32     'amazon', 'google', 'facebook', 'apple', 'microsoft',
33     'paypal', 'netflix', 'instagram', 'twitter', 'linkedin',
34     'vietcombank', 'vcb', 'bidv', 'techcombank', 'agribank',
35     'momo', 'zalopay', 'shopee', 'tiki', 'lazada', 'vpbank',
36     'mbbank', 'tpbank', 'sacombank', 'viettel', 'vinaphone'
37 }

```

```

38 SHORTENERS = {
39     'bit.ly', 'goo.gl', 'tinyurl.com', 't.co', 'ow.ly', 'is.gd',
40     'cutt.ly', 'rebrand.ly', 'bit.do', 'v.gd', 'short.to', 'tiny
    .cc'
41 }
42
43 # Free hosting platforms often used by attackers for phishing/
    malware distribution
44 FREE_HOSTING = {
45     'vercel.app', 'netlify.app', 'github.io', 'glitch.me',
46     'replit.dev', 'repl.co', '000webhostapp.com', 'web.app',
47     'firebaseapp.com', 'pages.dev', 'surge.sh', 'onrender.com',
48     'railway.app', 'fly.dev', 'cyclic.app'
49 }
50
51 # Executable / malware-distribution file extensions
52 EXECUTABLE_EXTENSIONS = {
53     '.exe', '.msi', '.bat', '.cmd', '.ps1', '.vbs', '.js', #
    Windows
54     '.sh', '.elf', '.bin', '.run', #
    Linux generic
55     '.i686', '.x86_64', '.aarch64', '.arm', #
    Linux arch-specific RPM/binary
56     '.rpm', '.deb', '.pkg', #
    Package managers
57     '.apk', '.ipa', #
    Mobile
58     '.dmg', '.app', #
    macOS
59     '.jar', '.class', #
    Java
60 }
61
62 def brand_similarity(sld: str) -> float:
63     max_score = 0.0
64     for brand in FAMOUS_BRANDS:
65         if brand in sld and sld != brand:
66             max_score = max(max_score, 0.85)
67             continue
68
69     len_ratio = min(len(sld), len(brand)) / max(len(sld),
    len(brand))
70     if len_ratio < 0.65:
71         continue
72     score = ((fuzz.ratio(sld, brand) + fuzz.partial_ratio(

```

```

sld, brand)) / 200.0) * len_ratio
73     max_score = max(max_score, score)
74
75     return round(max_score, 4)
76
77 _EXTRA_WHITELIST: set = set()
78 try:
79     with open('data/whitelist.txt', encoding='utf-8') as _f:
80         for _line in _f:
81             _d = _line.strip().lower()
82             if _d and not _d.startswith('#'):
83                 _EXTRA_WHITELIST.add(_d)
84             if len(_EXTRA_WHITELIST) >= 5000:
85                 break
86 except FileNotFoundError:
87     pass
88
89 def reload_extra_whitelist():
90     global _EXTRA_WHITELIST
91     _EXTRA_WHITELIST = set()
92     try:
93         with open('data/whitelist.txt', encoding='utf-8') as f:
94             for line in f:
95                 d = line.strip().lower()
96                 if d and not d.startswith('#'):
97                     _EXTRA_WHITELIST.add(d)
98                 if len(_EXTRA_WHITELIST) >= 5000:
99                     break
100     except FileNotFoundError:
101         pass
102
103 def is_trusted_domain(hostname: str) -> bool:
104     hostname = hostname.lower()
105     if hostname.startswith('www.'):
106         hostname = hostname[4:]
107     # Whitelist check
108     for trusted in TRUSTED_DOMAINS | _EXTRA_WHITELIST:
109         if hostname == trusted or hostname.endswith('.') +
110 trusted):
111             return True
112     # Real brand domain check using tldextract (handles eTLDs
113 like workers.dev, co.uk, com.vn)
114     ext = tldextract.extract(hostname)
115     sld = ext.domain
116     tld_leaf = ext.suffix.split('.')[0] if ext.suffix else ''

```

```

115     if sld in FAMOUS_BRANDS and tld_leaf not in SUSPICIOUS_TLDS:
116         return True
117     return False
118
119 def shannon_entropy(s):
120     if not s:
121         return 0.0
122     freq = {}
123     for c in s:
124         freq[c] = freq.get(c, 0) + 1
125     n = len(s)
126     return -sum((f / n) * math.log2(f / n) for f in freq.values
127                ())
128
129 def _subdomain_entropy(hostname: str) -> float:
130     # Entropy of subdomain to detect random-looking subdomains
131     ext = tldextract.extract(hostname)
132     subdomain = ext.subdomain
133     if not subdomain:
134         return 0.0
135     return round(shannon_entropy(subdomain), 4)
136
137 def _is_uuid_path(path: str, query: str) -> int:
138     # Detect if path or query contains a UUID pattern, which is
139     common in C2/malware callback URLs
140     uuid_pattern = r'[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}'
141     return 1 if re.search(uuid_pattern, path.lower()) or re.
142     search(uuid_pattern, query.lower()) else 0
143
144 def _subdomain_is_random(hostname: str) -> int:
145     ext = tldextract.extract(hostname)
146     subdomain = ext.subdomain
147     if not subdomain:
148         return 0
149     sub = subdomain.split('.')[0]
150     if len(sub) < 5 or len(sub) > 20:
151         return 0
152     vowels = sum(c in 'aeiou' for c in sub)
153     vowel_ratio = vowels / len(sub)
154     ent = shannon_entropy(sub)
155
156     no_vowels = vowel_ratio == 0.0 # No
157     vowels at all -> likely random
158     high_entropy = vowel_ratio < 0.30 and ent >= 2.4 # High

```

```

entropy + low vowel ratio -> likely random
155     return 1 if (no_vowels or high_entropy) else 0
156
157 def extracting_features(url):
158     url_lower = str(url).strip().lower()
159     url_lower = url_lower.rstrip('/')
160
161     clean_url = re.sub(r'^https?://', '', url_lower)
162     clean_url = re.sub(r'^www\.', '', clean_url)
163
164     try:
165         parsed = urlparse('//' + clean_url)
166     except Exception:
167         parsed = urlparse('/')
168
169     netloc = parsed.netloc or ''
170     path = parsed.path or ''
171     query = parsed.query or ''
172
173     hostname = netloc.split(':')[0] if ':' in netloc else netloc
174     has_port = 1 if re.search(r':\d+', netloc) else 0
175
176     # Use tldextract for accurate eTLD parsing
177     ext = tldextract.extract(hostname)
178     tld = ext.suffix
179     tld_leaf = tld.split('.')[0] if tld else ''
180     sld = ext.domain
181     subdomain_count = len(ext.subdomain.split('.')) if ext.subdomain else 0
182
183     n_chars = len(clean_url) if clean_url else 1
184     digit_ratio = sum(c.isdigit() for c in clean_url) / n_chars
185     letter_ratio = sum(c.isalpha() for c in clean_url) / n_chars
186
187     has_suspicious_word = 1 if any(w in url_lower for w in
188     SUSPICIOUS_WORDS) else 0
189     has_hex_chars = 1 if len(re.findall(r'%[0-9a-f]{2}',
190     url_lower)) > 2 else 0
191     consecutive_digits = 1 if re.search(r'\d{4,}', hostname)
192     else 0
193
194     brand_sim = brand_similarity(sld)
195
196     is_shortener = 1 if hostname in SHORTENERS or any(
197     hostname.endswith('.') + s for s in SHORTENERS) else 0

```

```

194     path_digit_ratio = sum(c.isdigit() for c in path) / len(path
195 ) if path else 0
196
197     subdomain_entropy = _subdomain_entropy(hostname)
198
199     has_uuid_path = _is_uuid_path(path, query)
200
201     subdomain_is_random = _subdomain_is_random(hostname)
202
203     is_free_hosting = 1 if any(hostname.endswith(h) for h in
204 FREE_HOSTING) else 0
205
206     has_download_param = 1 if (
207         'download' in query.lower() or
208         'download' in path.lower() or
209         'install' in path.lower()
210     ) else 0
211
212     free_hosting_download = 1 if (is_free_hosting and
213 has_download_param) else 0
214
215     # Detect executable/malware file extensions in path
216     import os
217     path_ext = os.path.splitext(path.lower())[1]
218     has_executable_ext = 1 if path_ext in EXECUTABLE_EXTENSIONS
219 else 0
220
221     features = {
222         'url_len': len(clean_url),
223         'domain_len': len(netloc),
224         'path_len': len(path),
225         'count_dot': clean_url.count('.'),
226         'count_hyphen': clean_url.count('-'),
227         'count_at': clean_url.count('@'),
228         'count_slash': clean_url.count('/'),
229         'count_question': clean_url.count('?'),
230         'count_equal': clean_url.count('='),
231         'is_ip': 1 if re.match(r'\d+\.\d+\.\d+\.\d+')
232 , hostname) else 0,
233         'shannon_entropy': round(shannon_entropy(hostname),
234 4),
235         'digit_ratio': round(digit_ratio, 4),
236         'letter_ratio': round(letter_ratio, 4),
237         'subdomain_count': subdomain_count,
238         'domain_digit_count': sum(c.isdigit() for c in hostname)
239     },

```



```

232         'tld_suspicious':      1 if tld_leaf in SUSPICIOUS_TLDS
else 0,
233         'count_percent':       clean_url.count('%'),
234         'count_ampersand':     query.count('&'),
235         'path_depth':          path.count('/'),
236         'has_port':             has_port,
237         'has_hex_chars':       has_hex_chars,
238         'has_suspicious_word':  has_suspicious_word,
239         'consecutive_digits':   consecutive_digits,
240         'brand_similarity':     brand_sim,
241         'is_shortener':         is_shortener,
242         'path_digit_ratio':     round(path_digit_ratio, 4),
243         'subdomain_entropy':    subdomain_entropy,
244         'has_uuid_path':        has_uuid_path,
245         'subdomain_is_random':  subdomain_is_random,
246         'is_free_hosting':      is_free_hosting,
247         'has_download_param':   has_download_param,
248         'free_hosting_download': free_hosting_download,
249         'has_executable_ext':   has_executable_ext,
250         'is_domain_only':       1 if (len(path) == 0 and not
query) else 0,
251     }
252     return features
253
254
255 def generate_whitelist(
256     src:      str = 'data/top-1m.csv',
257     dst:      str = 'data/whitelist.txt',
258     top_n:    int = 10000,
259     max_out:  int = 5000,
260 ):
261     count = 0
262     with open(src, encoding='utf-8') as f_in, \
263         open(dst, 'w', encoding='utf-8') as f_out:
264         f_out.write("# Auto-generated from Tranco top 1M\n")
265         for i, line in enumerate(f_in):
266             if i >= top_n or count >= max_out:
267                 break
268             line = line.strip()
269             if not line:
270                 continue
271             parts = line.split(',')
272             if len(parts) < 2:
273                 continue
274             domain = parts[1].lower()

```

```

275         tld      = domain.split('.')[ -1]
276         if tld in SUSPICIOUS_TLDS:
277             continue
278         f_out.write(domain + '\n')
279         count += 1
280     print(f"Done - {count} domains saved to {dst}")
281
282
283 if __name__ == "__main__":
284     import argparse
285     parser = argparse.ArgumentParser()
286     parser.add_argument('--src',      default='data/top-1m.csv')
287     parser.add_argument('--dst',      default='data/whitelist.txt
288 ')
289     parser.add_argument('--top_n',    type=int, default=10000)
290     parser.add_argument('--max_out',  type=int, default=5000)
291     args = parser.parse_args()
292     generate_whitelist(args.src, args.dst, args.top_n, args.
max_out)

```

Features are grouped into five categories described in the following subsections.

7.1 Structural & Length Features

These features capture the overall morphology of the URL and are strong baseline signals for randomness or obfuscation:

Feature	Description
url_len	Total character length of the cleaned URL (scheme and <code>www.</code> stripped).
domain_len	Length of the <code>netloc</code> component (<i>hostname:port</i>).
path_len	Length of the URL path component.
count_dot	Number of dots in the full URL (proxy for subdomain depth and TLD complexity).
count_hyphen	Number of hyphens; brand-spoof domains commonly insert hyphens (e.g., <code>paypal-secure.com</code>).
count_at	Presence of <code>@</code> in the URL; used to embed credentials before the real hostname.
count_slash	Slash count; deep paths are common in obfuscated redirect chains.
count_question	Number of <code>?</code> characters.
count_equal	Number of <code>=</code> characters (query parameter count proxy).
count_percent	Number of <code>%</code> characters; high values indicate heavy URL encoding / obfuscation.
count_ampersand	Number of <code>&</code> separators in the query string (parameter count).
path_depth	Number of <code>/</code> in the path; deep paths are unusual for legitimate landing pages.
has_port	Binary flag; non-standard ports (e.g., <code>:8080</code>) are uncommon for legitimate sites.
is_domain_only	Binary flag; set to 1 when the URL is a bare domain with no path and no query string. Helps the model learn that bare domains with clean signals are generally safe.

Table 3: Structural and length features

7.2 Statistical & Entropy Features

These features quantify the degree of randomness and character distribution within the URL. High entropy is a reliable proxy for machine-generated strings—a common property of DGA-based domains and obfuscated phishing hostnames—while digit and letter ratios expose structural anomalies that deviate from naturally readable domain names.

Feature	Description
shannon_entropy	Shannon entropy $H = -\sum p_i \log_2 p_i$ computed over the hostname characters. High entropy (typically > 3.5) indicates a randomly generated string.
subdomain_entropy	Entropy computed <i>exclusively</i> on the subdomain component (everything before SLD, TLD), isolating the random-subdomain signal without dilution from the base domain.
digit_ratio	Ratio of digit characters to total characters in the full URL.
letter_ratio	Ratio of alphabetic characters to total characters.
domain_digit_count	Absolute count of digits in the hostname.
consecutive_digits	Binary flag for 4+ consecutive digits in the hostname (e.g., 192login.com).
path_digit_ratio	Ratio of digits to total characters in the path component.

Table 4: Statistical and entropy features

7.3 Lexical & Heuristic Features

These features encode domain-expert knowledge about the vocabulary and structure of malicious URLs. Unlike the statistical group, they rely on curated reference sets—known bad TLDs, social-engineering keywords, famous brand names, and URL shortener hostnames—to produce binary signals that are both highly interpretable and computationally inexpensive.

Feature	Description
tld_suspicious	Binary flag; TLD membership in a curated set of high-abuse TLDs spanning generic abuse domains (.tk, .xyz, .top, .icu), gambling-related TLDs (.bet, .casino, .poker), and sport-themed TLDs (.football, .cricket, .golf).
has_suspicious_word	Binary flag; URL contains any of 15 social-engineering keywords (e.g., login, verify, password, prize).
has_hex_chars	Binary flag for more than 2 percent-encoded sequences (%XX); indicates character-level obfuscation.
is_ip	Binary flag; hostname matches an IPv4 address pattern. Legitimate services never use raw IPs as their primary URL.
subdomain_count	Number of subdomain levels above the SLD.
brand_similarity	Maximum fuzzy similarity score (0–1) between the SLD and any of 26 tracked brand names. The function first checks if the SLD <i>contains</i> a brand name as a substring (returning 0.85 if so), then falls back to the average of fuzz_ratio and fuzz_partial_ratio from RapidFuzz, scaled by a length ratio filter of ≥ 0.65 to suppress false positives from unrelated SLDs.
is_shortener	Binary flag; hostname matches a list of 12 known URL shortening services. Shorteners are used to obscure the final destination.

Table 5: Lexical and heuristic features

7.4 Advanced Cyber Threat Indicators

The following features address attack vectors not adequately captured by the baseline feature set:

Feature	Description
subdomain_is_random	Binary flag derived from two heuristics on the first subdomain label (length 5–20): (1) <i>zero vowels</i> (deterministic indicator of machine generation), or (2) vowel ratio < 0.30 combined with Shannon entropy ≥ 2.4 (probabilistic indicator).
has_uuid_path	Binary flag; path or query contains a UUID v4 pattern $[0-9a-f]\{8\}-[0-9a-f]\{4\}-[0-9a-f]\{4\}-[0-9a-f]\{4\}-[0-9a-f]\{12\}$. UUIDs in URL paths are a strong indicator of C2 server callbacks.
is_free_hosting	Binary flag; hostname ends with any of 15 known free developer hosting suffixes (e.g., <code>vercel.app</code> , <code>netlify.app</code> , <code>github.io</code> , <code>000webhostapp.com</code>).
has_download_param	Binary flag; the path or query string contains the keywords <code>download</code> or <code>install</code> .
free_hosting_download	Composite flag: <code>is_free_hosting</code> AND <code>has_download_param</code> . Direct trigger for Hard Rule 7.

Table 6: Advanced cyber threat indicator features

7.5 Executable File Detection Features

The following features were introduced to detect malware distribution campaigns that serve executable or installable files directly via URLs:

Feature	Description
has_executable_ext	Binary flag; the URL path ends with a known executable or installable file extension. The system maintains a curated set of extensions covering Windows (<code>.exe</code> , <code>.msi</code> , <code>.bat</code> , <code>.cmd</code> , <code>.ps1</code> , <code>.vbs</code> , <code>.js</code>), Linux (<code>.sh</code> , <code>.elf</code> , <code>.bin</code> , <code>.run</code> , <code>.deb</code> , <code>.rpm</code>), macOS (<code>.dmg</code> , <code>.app</code>), mobile (<code>.apk</code> , <code>.ipa</code>), Java (<code>.jar</code> , <code>.class</code>), and architecture-specific binaries (<code>.i686</code> , <code>.x86_64</code> , <code>.aarch64</code> , <code>.arm</code>).

Table 7: Executable file detection feature

7.6 Feature Summary

Table 8: Summary of the five feature groups used in Binary Classification.

Group	Category	Count
1	Structural & Length	14
2	Statistical & Entropy	7
3	Lexical & Heuristic	7
4	Advanced Threat Indicators	5
5	Executable File Detection	1
Total		34

8 Model Training

Model training is encapsulated in `training.py`, which handles the full pipeline from raw CSV ingestion to serialized model artifacts. The complete implementation is listed below:

Listing 3: `training.py` — Full training pipeline (data loading, RF, XGB, LGB & LR training, ensemble weight search)

```
1 import pandas as pd
2 import numpy as np
3 import xgboost as xgb
4 import lightgbm as lgb
5 from sklearn.ensemble import RandomForestClassifier
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.pipeline import make_pipeline
9 from sklearn.metrics import classification_report, fbeta_score
10 from sklearn.model_selection import train_test_split
11 import joblib
12 import json
13 import time
14 from feature import extracting_features
15
16 def find_best_weights(rf_model, xgb_model, lgb_model, lr_model,
17                      X_test, y_test):
18     probs_rf = rf_model.predict_proba(X_test)[: , 1]
19     probs_xgb = xgb_model.predict_proba(X_test)[: , 1]
20     probs_lgb = lgb_model.predict_proba(X_test)[: , 1]
21     probs_lr = lr_model.predict_proba(X_test)[: , 1]
22
23     best_weights, best_f2 = (0.25, 0.25, 0.25, 0.25), 0
```

```

24     # Grid search over 4 weights with step 0.1, constrained to
    sum = 1.0
25     for w_rf in np.arange(0.0, 1.1, 0.1):
26         for w_xgb in np.arange(0.0, 1.1 - w_rf, 0.1):
27             for w_lgb in np.arange(0.0, 1.1 - w_rf - w_xgb, 0.1)
:
28                 w_lr = round(1.0 - w_rf - w_xgb - w_lgb, 1)
29                 if w_lr < -0.01:
30                     continue
31                 w_lr = max(w_lr, 0.0)
32
33                 avg_prob = w_rf * probs_rf + w_xgb * probs_xgb +
w_lgb * probs_lgb + w_lr * probs_lr
34                 preds = (avg_prob >= 0.5).astype(int)
35                 f2 = fbeta_score(y_test, preds, beta=2.0,
pos_label=1)
36                 if f2 > best_f2:
37                     best_f2 = f2
38                     best_weights = (round(w_rf, 1), round(w_xgb,
1), round(w_lgb, 1), round(w_lr, 1))
39
40                 print(f"Best weights: w_rf={best_weights[0]} | w_xgb={
best_weights[1]} | w_lgb={best_weights[2]} | w_lr={
best_weights[3]} | F2: {best_f2:.4f}")
41                 return best_weights
42
43 def run_training():
44     df = pd.read_csv('data/malicious_phish.csv', encoding='latin
-1')
45     df['label'] = np.where(df['type'] == 'benign', 0, 1)
46     df['url'] = df['url'].fillna('')
47
48     list_of_dicts = df['url'].apply(extracting_features).tolist
()
49     X = pd.DataFrame(list_of_dicts)
50     y = df['label']
51
52     X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=0)
53
54     # --- Random Forest ---
55     print("Training Random Forest...")
56     rf_model = RandomForestClassifier(
57         n_estimators=200, max_depth=20,
58         class_weight={0: 1, 1: 2.0},

```



```

59         n_jobs=-1, random_state=0
60     )
61     start = time.time()
62     rf_model.fit(X_train, y_train)
63     print(f"Training time: {time.time() - start:.2f}s")
64     joblib.dump(rf_model, 'models/rf_model.pkl')
65     print(classification_report(y_test, rf_model.predict(X_test)
, target_names=['safe', 'dangerous'], digits=4))
66
67     # --- XGBoost ---
68     print("Training XGBoost...")
69     ratio = float(np.sum(y_train == 0)) / np.sum(y_train == 1)
70     xgb_model = xgb.XGBClassifier(
71         n_estimators=200, max_depth=10, learning_rate=0.1,
72         scale_pos_weight=ratio * 1.5,
73         n_jobs=-1, random_state=0
74     )
75     start = time.time()
76     xgb_model.fit(X_train, y_train)
77     print(f"Training time: {time.time() - start:.2f}s")
78     xgb_model.save_model('models/xgb_model.json')
79     print(classification_report(y_test, xgb_model.predict(X_test)
, target_names=['safe', 'dangerous'], digits=4))
80
81     # --- LightGBM ---
82     print("Training LightGBM...")
83     lgb_model = lgb.LGBMClassifier(
84         n_estimators=200, num_leaves=63, max_depth=-1,
85         learning_rate=0.1,
86         scale_pos_weight=ratio * 1.5,
87         n_jobs=-1, random_state=0, verbose=-1
88     )
89     start = time.time()
90     lgb_model.fit(X_train, y_train)
91     print(f"Training time: {time.time() - start:.2f}s")
92     joblib.dump(lgb_model, 'models/lgb_model.pkl')
93     print(classification_report(y_test, lgb_model.predict(X_test)
, target_names=['safe', 'dangerous'], digits=4))
94
95     # --- Logistic Regression ---
96     print("Training Logistic Regression (Baseline)...")
97     lr_model = make_pipeline(
98         StandardScaler(),
99         LogisticRegression(max_iter=1000, class_weight='balanced'
, n_jobs=-1, random_state=0)

```

```

100     )
101     start = time.time()
102     lr_model.fit(X_train, y_train)
103     print(f"Training time: {time.time() - start:.2f}s")
104     joblib.dump(lr_model, 'models/lr_model.pkl')
105     print(classification_report(y_test, lr_model.predict(X_test)
106                                , target_names=['safe', 'dangerous'], digits=4))
107
108     # --- Ensemble Weight Search ---
109     print("\n=== FIND ENSEMBLE WEIGHTS ===")
110     w_rf, w_xgb, w_lgb, w_lr = find_best_weights(rf_model,
111                                                  xgb_model, lgb_model, lr_model, X_test, y_test)
112     json.dump({'w_rf': w_rf, 'w_xgb': w_xgb, 'w_lgb': w_lgb, '
113              w_lr': w_lr}, open('models/weights.json', 'w'))
114
115     probs_rf = rf_model.predict_proba(X_test)[: , 1]
116     probs_xgb = xgb_model.predict_proba(X_test)[: , 1]
117     probs_lgb = lgb_model.predict_proba(X_test)[: , 1]
118     probs_lr = lr_model.predict_proba(X_test)[: , 1]
119     avg_prob = w_rf * probs_rf + w_xgb * probs_xgb + w_lgb *
120               probs_lgb + w_lr * probs_lr
121     print("\n=== ENSEMBLE (RF + XGB + LGB + LR) ===")
122     print(classification_report(y_test, (avg_prob >= 0.5).astype
123                                     (int), target_names=['safe', 'dangerous'], digits=4))
124
125 if __name__ == "__main__":
126     run_training()

```

8.1 Data Loading and Label Aggregation

The dataset is loaded from `data/malicious_phish.csv` and immediately subjected to binary label aggregation: any URL whose type column is not benign is assigned label 1 (Dangerous); the rest receive label 0 (Safe). Feature extraction is then applied to every URL via `extracting_features()`, producing a 34-column DataFrame `X`. An 80/20 stratified split (`random_state=0`) is used consistently for all four models.

8.2 Random Forest Hyperparameters

Parameter	Value / Rationale
n_estimators	200 — sufficient ensemble size for stable variance reduction
max_depth	20 — deep enough to capture complex feature interactions
class_weight	{0: 1, 1: 2.0} — penalizes false negatives on the dangerous class
n_jobs	−1 — uses all available CPU cores
random_state	0 — ensures reproducibility
Training time	116.61 s (on full dataset, single machine)

Table 9: Random Forest hyperparameters (RandomForestClassifier)

8.3 XGBoost Hyperparameters

Parameter	Value / Rationale
n_estimators	200
max_depth	10
learning_rate	0.1
scale_pos_weight	$\text{ratio} \times 1.5$, where $\text{ratio} = \text{benign} / \text{malicious} $ — amplified class imbalance correction biased toward recall
n_jobs	−1
random_state	0
Training time	14.70 s (on full dataset, single machine)

Table 10: XGBoost hyperparameters (XGBClassifier)

The `scale_pos_weight` multiplier of $1.5\times$ above the natural class ratio is a deliberate over-correction: in a security context, a false negative (missed threat) is considered substantially more costly than a false positive.

8.4 LightGBM Hyperparameters

Parameter	Value / Rationale
n_estimators	200
num_leaves	63 — leaf-wise growth; controls model complexity more directly than <code>max_depth</code>
max_depth	−1 (unlimited) — depth is regulated by <code>num_leaves</code> instead
learning_rate	0.1
scale_pos_weight	$\text{ratio} \times 1.5$ — same imbalance correction as XGBoost
n_jobs	−1
random_state	0
Training time	7.40 s (on full dataset, single machine)

Table 11: LightGBM hyperparameters (LGBMClassifier)

Unlike XGBoost’s level-wise tree growth, LightGBM employs a **leaf-wise** strategy: at each iteration, it expands only the leaf with the largest loss reduction, producing deeper but more asymmetric trees. This approach converges faster and often yields better accuracy on large datasets, but requires careful tuning of `num_leaves` to prevent overfitting. The `verbose = -1` flag suppresses per-iteration logging output.

LightGBM’s training time (7.40 s) is roughly half that of XGBoost (14.70 s) and an order of magnitude faster than Random Forest (116.61 s), making it the fastest model in the ensemble.

8.5 Ensemble Weight Search via F_2 Score

After all four models are trained independently, the `find_best_weights()` function performs a 4-way grid search over $w_{\text{RF}}, w_{\text{XGB}}, w_{\text{LGB}}, w_{\text{LR}} \in \{0.0, 0.1, \dots, 1.0\}$ subject to the constraint $w_{\text{RF}} + w_{\text{XGB}} + w_{\text{LGB}} + w_{\text{LR}} = 1.0$, to find the weighting that maximizes the F_2 score on the held-out test set.

The F_2 score is defined as:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \times \text{Recall}}{(\beta^2 \cdot \text{Precision}) + \text{Recall}}, \quad \beta = 2$$

With $\beta = 2$, recall is weighted *twice* as heavily as precision. This reflects the asymmetric cost structure of the domain: failing to detect a phishing URL (false negative) is considered twice as harmful as incorrectly flagging a safe URL (false positive). The optimal weights are serialized to `models/weights.json` and loaded by `predict.py` at inference time.

Observed Result. In the current deployment, the grid search converged to $w_{\text{rf}} = 0.0$, $w_{\text{xgb}} = 1.0$, $w_{\text{lgb}} = 0.0$, and $w_{\text{lr}} = 0.0$, meaning the system relies *exclusively* on XGBoost for the probability estimation, with Random Forest, LightGBM, and Logistic Regression all receiving zero weight. This outcome is consistent with XGBoost’s superior F_2 score on the held-out test set. All four models remain loaded and available for ensemble rebalancing should the training data distribution change in future iterations.

9 Ensemble Learning Model

The system combines the power of Tree-based Machine Learning models alongside a linear baseline, optimized via a Weighted Average approach. Although the F_2 -optimized weight search currently assigns all weight to XGBoost ($w_{\text{rf}} = 0.0$, $w_{\text{xgb}} = 1.0$, $w_{\text{lgb}} = 0.0$, $w_{\text{lr}} = 0.0$), the ensemble infrastructure is retained for future rebalancing as the training data evolves.

9.1 Random Forest (RF)

Representing the “Democratic” (Bagging) mindset, the Random Forest model builds hundreds of independent decision trees. It excels at recognizing valid, complex web structures (such as SEO-friendly news URLs) and is highly resistant to overfitting.

Class	Precision	Recall	F1-Score	Support
Benign (0)	0.9167	0.8491	0.8816	85,986
Phishing (1)	0.7459	0.8515	0.7952	44,712
Accuracy	85.00%			130,698
Training time	116.61 s			

Table 12: Classification report and training time — Random Forest

9.2 XGBoost

Representing the “Assassin” (Boosting) mindset, XGBoost continuously corrects previous mispredictions. It is extremely sensitive to anomalous structures and sophisticated evasion tactics (e.g., Hex encoding, IP obfuscation). Skipping data scaling for XGBoost significantly speeds up training without compromising its predictive accuracy.

Class	Precision	Recall	F1-Score	Support
Benign (0)	0.9439	0.7975	0.8645	85,986
Phishing (1)	0.7000	0.9088	0.7908	44,712
Accuracy	83.55%			130,698
Training time	14.70 s			

Table 13: Classification report and training time — XGBoost

9.3 LightGBM

Representing the “Precision Surgeon” (Leaf-wise Boosting) mindset, LightGBM grows trees by expanding only the leaf with the highest loss reduction at each step. This leaf-wise strategy produces deeper, more asymmetric trees that converge faster than XGBoost’s level-wise approach. LightGBM also employs Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB) to accelerate training on large datasets while maintaining accuracy.

Class	Precision	Recall	F1-Score	Support
Benign (0)	0.9376	0.7574	0.8379	85,986
Phishing (1)	0.6593	0.9031	0.7622	44,712
Accuracy	80.72%			130,698
Training time	7.40 s			

Table 14: Classification report and training time — LightGBM

9.4 Logistic Regression (Baseline)

Serving as the linear baseline, Logistic Regression provides a point of comparison against the complex Tree-based models. While it scales efficiently to large datasets, its linear decision boundary struggles to capture the complex, non-linear relationships among lexical features (e.g., the interaction between suspicious TLDs and brand similarity). It is included primarily to benchmark the performance gains achieved by the tree-based ensemble techniques.

Class	Precision	Recall	F1-Score	Support
Benign (0)	0.7819	0.7778	0.7798	85,986
Phishing (1)	0.5769	0.5828	0.5798	44,712
Accuracy	71.11%			130,698
Training time	5.33 s			

Table 15: Classification report and training time — Logistic Regression

9.5 Weighted Average Ensemble Strategy

The final prediction utilizes a weighted ensemble mechanism across all four models. The system assigns weights based on F_2 optimization on the validation set. The weights are stored externally in `models/weights.json` and loaded at startup:

Listing 4: Loading model weights from `weights.json`

```

1 _w = json.load(open('models/weights.json'))
2 W_RF, W_XGB, W_LGB, W_LR = _w['w_rf'], _w['w_xgb'], _w.get('
   w_lgb', 0.0), _w.get('w_lr', 0.0)
3
4 # Weighted probability ensemble
5 prob = W_RF * p_rf + W_XGB * p_xgb + W_LGB * p_lgb + W_LR * p_lr

```

In the current deployment, the grid search yielded $w_{rf} = 0.0$, $w_{xgb} = 1.0$, $w_{lgb} = 0.0$, and $w_{lr} = 0.0$, effectively making XGBoost the sole decision-maker in the

probability computation. This result reflects XGBoost’s superior F_2 score, which balances recall and precision most favorably among the models. The multi-model ensemble architecture is nonetheless preserved: should the data distribution shift, re-running the weight search can automatically reintroduce other models’ contributions without code changes.

9.6 Method Comparison

The four models exhibit distinct training time profiles: Random Forest requires 116.61 s due to constructing 200 deep independent trees in parallel, XGBoost completes in 14.70 s using sequential level-wise boosting, LightGBM finishes in 7.40 s owing to its leaf-wise growth strategy, and Logistic Regression converges in just 5.33 s as it only fits a single linear boundary.

Comparative Analysis. A closer inspection of the four classification reports reveals a nuanced hierarchy. Random Forest achieves the highest overall accuracy (85.00%), but this aggregate figure is misleading: it is sustained by balanced performance across both classes. When the focus shifts to the Dangerous class—the class that actually matters for security—XGBoost leads with a recall of **0.9088**, followed closely by LightGBM at **0.9031**, and then Random Forest at **0.8515**. Logistic Regression trails far behind with a recall of only **0.5828** and an overall accuracy of 71.11%, definitively validating the necessity of complex, non-linear ensemble techniques for this domain.

LightGBM’s recall (0.9031) is nearly identical to XGBoost’s, but its precision on the Dangerous class is substantially lower (0.6593 vs. 0.7000), indicating more false positives. This precision gap explains why the F_2 weight search assigned zero weight to LightGBM despite its competitive recall: the additional false alarms outweigh the marginal recall gain when combined with XGBoost.

This pattern is a direct consequence of the boosting objective: both XGBoost and LightGBM iteratively re-weight misclassified samples, which in a security-skewed dataset means they aggressively pursue the harder-to-detect malicious examples. Random Forest, by democratic averaging, tends to be pulled toward the dominant (Benign) class. XGBoost’s level-wise strategy produces more balanced trees that generalize better on this specific dataset, while LightGBM’s leaf-wise approach, though faster, tends to overfit slightly to noisy patterns—manifesting as lower precision. These characteristics explain why the F_2 -optimized weight search assigned 100% weight to XGBoost, as formalized in Section 16.

10 Dynamic Thresholding Mechanism

After the ML ensemble produces a weighted probability score $p \in [0, 1]$, the system determines the classification threshold dynamically rather than applying a fixed cut-off.

The threshold is selected by evaluating six mutually exclusive contextual conditions derived from the URL structure, query string content, and domain trustworthiness.

10.1 Context Signals

Before the threshold rules are applied, the following boolean signals are computed:

- `q_count`: Number of query parameters (obtained by splitting the query string on `&`).
- `has_tracking`: True if the query string contains any of the 38 analytics/affiliate tracking parameters listed in Table 16, grouped across UTM, click IDs, email marketing, and affiliate categories.
- `is_short_sus`: True if the URL is from a known shortening service (`is_shortener = 1`), or if the URL is shorter than 30 characters with a high digit ratio in the path (`path_digit_ratio > 0.3`).
- `is_word_salad`: True if the URL exceeds 60 characters *and* contains a suspicious keyword. This pattern is characteristic of auto-generated phishing pages that embed keywords for SEO manipulation.
- `url_long_clean`: True if the URL is long (`url_len > 80`), carries multiple query parameters (`q_count > 3`), has at least one tracking parameter, uses a non-suspicious TLD, and does not resolve to a raw IP address. This pattern is typical of legitimate marketing campaigns or e-commerce deep links.
- `clean_vn`: True if the hostname has a Vietnamese eTLD (`.vn`, `.com.vn`, `.gov.vn`, etc.) and simultaneously satisfies: brand similarity < 0.75 , no suspicious words, no raw IP, no random subdomain, no free hosting, and no executable extension. See Section 10.3.
- `clean_domain`: True if the URL uses a non-suspicious TLD and satisfies: brand similarity < 0.65 , no suspicious words, no raw IP, no random subdomain, no free hosting, and no executable extension.

Parameter(s)	Origin / Purpose
<i>UTM / Google Analytics</i> utm_source, utm_medium, utm_campaign, utm_term, utm_content, utm_id	Standard Google Analytics campaign attribution parameters
<i>Click IDs</i> gclid, wbraid, gbraid, dclid fbclid, igshid msclkid ttclid twclid yclid li_fat_id	Google Ads click / conversion identifiers Facebook & Instagram click identifiers Microsoft Ads (Bing) click identifier TikTok Ads click identifier Twitter / X Ads click identifier Yandex Ads click identifier LinkedIn Ads click identifier
<i>Email Marketing</i> mc_cid, mc_eid mkt_tok hsCtaTracking	Mailchimp campaign & email identifiers Marketo tracking token HubSpot CTA tracking token
<i>Affiliate & Generic</i> aff_id, affid, click_id, clickid, cid, subid, tag sp_atk, skuid ref, aid, label	Generic affiliate / referral identifiers Shopee affiliate key & SKU identifier Generic referral parameters
<i>Analytics Helpers</i> _ga, _gl, s_kwcid, gad_source PHPSESSID, JSESSIONID	Google Analytics client ID, cross-domain linker, keyword & ad source Server session tokens (appear in marketing redirect chains)

Table 16: Tracking parameters used to identify legitimate marketing URLs (TRACKING_PARAMS), 38 entries grouped by category

10.2 Threshold Selection Rules

The six conditions are evaluated in priority order. The first matching condition determines the threshold:

Priority	Threshold	Condition and Rationale
1 (Highest)	0.30	Suspicious short URL (<code>is_short_sus</code>). URL shorteners conceal malicious destinations; even low model confidence should trigger a warning.
2	0.35	Keyword-stuffed long URL (<code>is_word_salad</code>). URLs > 60 characters with social-engineering keywords are highly characteristic of phishing pages.
3	0.60	Long URL with tracking parameters (<code>url_long_clean</code>). Long URLs with UTM/affiliate parameters and a reputable TLD strongly suggest legitimate marketing links; threshold raised to avoid false positives.
4	0.92	Clean Vietnamese domain (<code>clean_vn</code>). Vietnamese eTLDs are nationally regulated by VNNIC; domains passing all safety checks receive a very high threshold (see Section 10.3).
5	0.85	Clean generic domain (<code>clean_domain</code>). Non-suspicious TLD with no brand spoof, no suspicious words, no IP, no random subdomain, no free hosting, and no executable extension—threshold raised to protect reputable generic domains.
6 (Default)	0.45	General case. Slightly below 0.5 to bias toward recall (reducing false negatives) at an acceptable precision cost.

Table 17: Dynamic threshold selection rules in priority order

10.3 Vietnamese Domain Protection (`clean_vn`)

A separate conditional block raises the threshold to 0.92 for URLs identified as clean Vietnamese domains. A domain qualifies as `clean_vn` if it satisfies all of the following conditions simultaneously:

- Hostname has a Vietnamese effective TLD (one of `.vn`, `.com.vn`, `.net.vn`, `.org.vn`, `.edu.vn`, `.gov.vn`, `.int.vn`, `.ac.vn`, `.biz.vn`, `.info.vn`, `.name.vn`, `.pro.vn`), determined via `tldextract`
- `brand_similarity < 0.75` (not a clear brand spoof attempt)
- `has_suspicious_word = 0` (no social-engineering keywords)
- `is_ip = 0` (not a raw IP address)
- `subdomain_is_random = 0` (subdomain is human-readable)
- `is_free_hosting = 0` (not on a free hosting platform)
- `has_executable_ext = 0` (no executable file in path)

This rule acknowledges that the Vietnamese `.vn` namespace is nationally regulated and

requires identity verification for registration, making it substantially less susceptible to abuse than generic TLDs. The high threshold of 0.92 reflects a strong prior belief in the domain’s legitimacy, requiring near-certainty from the ML model before overriding that trust.

10.4 Clean Generic Domain Protection (`clean_domain`)

Additionally, a `clean_domain` threshold of 0.85 protects non-Vietnamese domains that pass a similar but slightly more conservative set of checks: `tld_suspicious = 0`, `brand_similarity < 0.65`, `has_suspicious_word = 0`, `is_ip = 0`, `subdomain_is_random = 0`, `is_free_hosting = 0`, and `has_executable_ext = 0`. This prevents false positives on legitimate international domains (e.g., news portals, SaaS products) that the model may score moderately due to long or complex URL structures.

11 Limitations and Future Work

While the system achieves robust performance in its intended operational context, several structural and methodological limitations must be acknowledged.

11.1 Lexical-Only Analysis

The system’s core design choice—analyzing *only* the URL string without fetching any page content—delivers sub-millisecond latency and zero external dependencies, but fundamentally limits detection capability:

- **Content-agnostic evasion:** Attackers can host phishing pages on domains with perfectly benign-looking URL structures (e.g., a cloned login page hosted at `example-store.com/products/new`). Since the system never examines HTML, JavaScript, or visual elements, such attacks are invisible to it.
- **Compromised legitimate domains:** If an attacker injects a phishing page into a legitimate, whitelisted domain (e.g., via a subdirectory on a compromised WordPress site), the whitelist check will bypass all further analysis, resulting in a false negative.
- **No redirect chain resolution:** The system cannot follow HTTP redirects. A malicious URL that appears safe lexically but redirects through multiple hops to a phishing page will not be caught.

11.2 Training Data Staleness

The model is trained on a static snapshot of malicious URLs from the Kaggle dataset and the Tin Nhiem Mang crawl. Phishing campaigns evolve rapidly—new attack vectors, novel brand impersonation techniques, and freshly registered TLDs emerge

continuously. Without periodic retraining or online learning, the model’s effectiveness degrades over time as it encounters URL patterns absent from its training distribution (**concept drift**).

11.3 Sensitivity to URL Normalization

As observed during development, the prediction output can be sensitive to minor variations in URL formatting (e.g., the presence or absence of a trailing slash, scheme differences between `http://` and `https://`). Although normalization logic was implemented to mitigate this (stripping trailing slashes, forcing `https://`), edge cases remain where cosmetically identical URLs produce different feature vectors—particularly in features like `path_len`, `path_depth`, and `count_slash`.

11.4 Over-Reliance on Hard Rules

The deterministic hard rule engine provides high-confidence verdicts but introduces rigidity:

- **False positives from substring brand matching:** The `brand_similarity` function’s `contains-check` (returning 0.85 if the SLD contains a brand name as a substring) can flag legitimate services whose names coincidentally embed a brand string (e.g., a domain containing “apple” in an unrelated context).
- **Rule ordering dependency:** Since rules are evaluated sequentially with first-match termination, the order of rules implicitly encodes priority assumptions. A URL matching multiple rules may be classified differently depending on evaluation order, and this ordering is manually tuned rather than learned.

11.5 Geographic and Linguistic Bias

The system is optimized for English-language URLs and Vietnamese domains. Phishing campaigns targeting other languages (e.g., Cyrillic homograph attacks, Arabic script obfuscation) or exploiting non-Latin IDN domains are not addressed by the current feature set. The `FAMOUS_BRANDS` and `SUSPICIOUS_WORDS` lists are curated for the English/Vietnamese market, leaving gaps for regional attacks in other geographies.

11.6 Ensemble Collapse

The F_2 -optimized weight search converged to $w_{\text{rf}} = 0.0$, $w_{\text{xgb}} = 1.0$, $w_{\text{lgb}} = 0.0$, and $w_{\text{lr}} = 0.0$, meaning Random Forest, LightGBM, and Logistic Regression contribute *nothing* to the final probability despite being loaded and maintained. This represents:

- **Reduced model diversity:** A true ensemble derives its strength from combining diverse learners. With only one model active out of four, the system loses the

variance-reduction benefit and becomes fully dependent on XGBoost's specific learned decision boundaries.

- **Single point of failure:** Any systematic bias or blind spot in XGBoost (e.g., a class of URLs it consistently misclassifies) is not corrected by complementary models.
- **Wasted computational resources:** The Random Forest model occupies ≈ 414 MB on disk, while LightGBM and Logistic Regression add further overhead; all three are loaded into memory at startup, consuming resources without contributing to inference.

11.7 Dynamic Threshold Fragility

The six-level threshold cascade relies on manually tuned constants (0.30, 0.35, 0.45, 0.60, 0.85, 0.92) and boolean heuristics. These thresholds were empirically calibrated on the current dataset but may be suboptimal for:

- Adversarial URLs specifically crafted to fall just below a threshold boundary.
- URL populations with different base rates of malicious content than the training set.
- Newly emerging URL patterns (e.g., AI-generated domain names) that do not match any of the predefined context signals.

11.8 Whitelist Maintenance Burden

The whitelist depends on a static `data/whitelist.txt` file generated from the Tranco Top-1M list, which is a point-in-time snapshot. Domains can:

- **Rise in popularity** after the snapshot and remain un-whitelisted, leading to false positives.
- **Be compromised** after being whitelisted, creating a permanent bypass for attackers who hijack or acquire trusted domains.
- **Expire and be re-registered** by malicious actors (expired domain takeover), inheriting the trust of their previous owners.

11.9 Future Work

To address the aforementioned limitations and enhance the system's robustness, several avenues for future research and engineering are proposed:

- **Content-Aware Fallback:** Integrating a lightweight HTML/DOM parser or fetching page screenshots for URLs that score in the borderline confidence range, mitigating the blind spots of purely lexical analysis.

- **Continuous Learning Pipeline:** Implementing an automated retraining loop (MLOps) that periodically ingests fresh threat intelligence feeds and newly discovered phishing URLs to combat concept drift.
- **Dynamic Threshold Calibration:** Replacing manually tuned thresholds with adaptive boundary mechanisms or calibrating probabilities using Platt Scaling to optimize the false-positive/false-negative trade-off dynamically.
- **Model Pruning and Distillation:** Given the observed ensemble collapse, transitioning to a single optimized model (e.g., XGBoost) or distilling the ensemble into a lighter model to recover wasted computational overhead.
- **Automated Whitelist Curation:** Linking the whitelist to real-time APIs (e.g., Tranco or Cloudflare Radar) to automatically age out inactive domains and quickly trust newly popular legitimate sites.
- **Brand Similarity Refinement:** Replacing the current substring-based contains-check with context-aware matching (e.g., using token boundaries or edit-distance thresholds) to reduce false positives caused by coincidental brand-name substrings in unrelated domains, while preserving high recall against genuine brand-spoofing attacks.
- **Dynamic Tranco Integration:** Transitioning from a static point-in-time Tranco snapshot to a periodically refreshed feed, incorporating domain age verification and compromise detection to prevent expired or hijacked domains from retaining permanent whitelist trust.

12 Conclusion

This system blends security domain knowledge with AI generalization. The modular architecture ensures it is ready for real-time API integration and global scalability. However, as outlined in Section 11, its lexical-only approach, static training data, and reliance on manually tuned heuristics impose inherent ceilings on detection capability. The proposed future work aims to address these bottlenecks, maintaining and extending the system’s effectiveness against evolving threats.

Part III

Multi-class Classification via Static Lexical Analysis

13 Training Data Sources

13.1 Primary Dataset: Kaggle Malicious Phish Corpus

The primary training corpus is sourced from the publicly available *Malicious Phish* dataset on Kaggle, comprising **653,490** labelled URL samples. Each record consists of two fields: the raw URL string and its associated threat category (benign, defacement, malware, or phishing). The dataset exhibits moderate class imbalance, with the benign class constituting the majority.

13.2 Supplementary Dataset: URLhaus Malware Feed

To augment the representation of the malware class, an additional **18,696** URLs are incorporated from the URLhaus threat intelligence feed maintained by abuse.ch [4]. These URLs are uniformly assigned the malware label. To prevent record duplication, only URLs absent from the primary dataset are appended.

Listing 5: Data aggregation procedure for URLhaus supplementary URLs

```
1 df_urlhaus = pd.read_csv(urlhaus_path, skiprows=8)
2 new_urls = ~df_urlhaus['url'].isin(url_data['url'])
3 df_new = df_urlhaus[new_urls][['url']].copy()
4 df_new['type'] = 'malware'
5 df_updated = pd.concat([url_data, df_new], ignore_index=True)
```

Following aggregation, the combined corpus contains **672,186** raw samples. After deduplication by URL string, the final working dataset comprises **661,976** unique samples.

13.3 Whitelist Reference: Trusted Domain Catalogue

A curated whitelist of **10,001 authoritative domains** is maintained to support the Levenshtein similarity computation described in Section 15.5.1. This catalogue encompasses globally prominent domains (e.g., google.com, facebook.com, microsoft.com), Vietnamese institutional and financial domains (e.g., vietcombank.com.vn, vnexpress.net), and governmental or intergovernmental entities (e.g., chinhphu.vn, ieee.org, nih.gov).

14 Processing Pipeline

The system operates under an *offline training, online inference* paradigm. The training pipeline transforms raw URL strings into a feature matrix, trains the LightGBM classifier, and persists the model artifacts. The inference pipeline accepts a single URL, extracts an identical feature vector, and returns class-conditional probabilities. Figure 2 illustrates both pipelines.

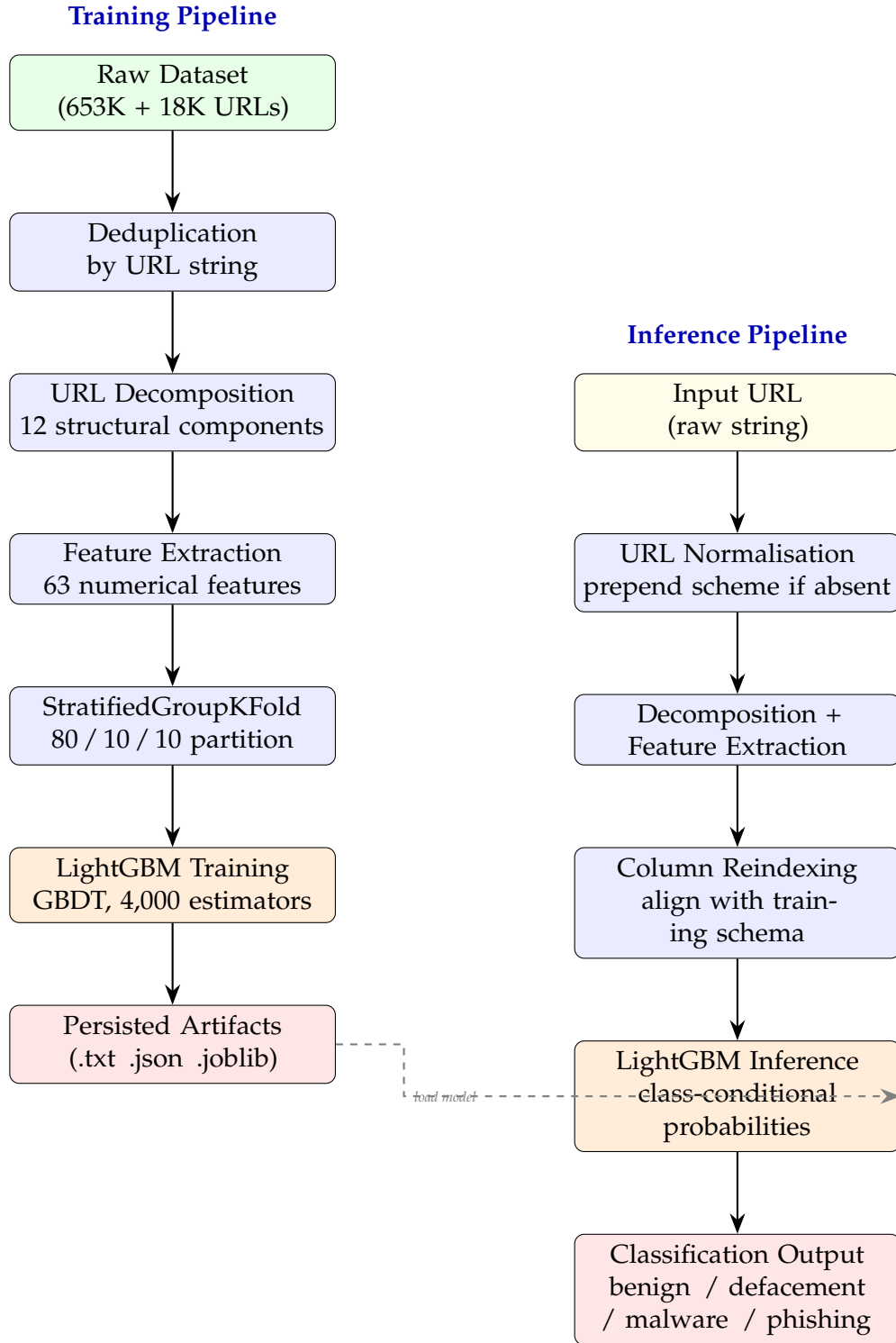


Figure 2: High-level architecture of the training and inference pipelines.

14.1 URL Normalisation

Raw URLs in the wild frequently omit the scheme component (e.g., `example.com/path` rather than `https://example.com/path`). To ensure consistent parsing, a two-stage normalisation procedure is applied:

1. **Training-time normalisation.** If the URL does not match the pattern $^[a-zA-Z][a-zA-Z0-9+.*://$ and does not begin with `//`, the prefix `//` is prepended. This enables `urlparse` to correctly identify the network location without imposing an artificial scheme.
2. **Inference-time normalisation.** For user-submitted URLs that lack both a scheme and any path separator (`/`), the prefix `https://` is prepended as a sensible default.

Listing 6: URL normalisation functions

```

1 def normalize_url(url: str) -> str:
2     """Training-time normalisation."""
3     if re.match(r'^[a-zA-Z][a-zA-Z0-9+\-\.\.]*://', url):
4         return url                                # scheme already present
5     if url.startswith('//'):
6         return url                                # protocol-relative URL
7     return '//'+url                               # prepend // for urlparse
8
9 def normalize_input_url(url: str) -> str:
10    """Inference-time normalisation."""
11    url = url.strip()
12    if not re.match(r'^[a-zA-Z][a-zA-Z0-9+\-\.\.]*://', url) \
13        and '/' not in url:
14        url = 'https://' + url
15    return url

```

14.2 Inference Pipeline

At inference time, the following procedure is executed for each input URL:

1. **Artifact loading.** The persisted LightGBM Booster (`.txt`), the label encoder (`.joblib`), the metadata schema (`.json`), and the whitelist are loaded into memory.
2. **Normalisation.** The input URL is normalised as described above.
3. **Feature extraction.** The function `features_extraction(url, whitelist)` computes all 63 features.
4. **Post-processing.** Raw textual columns (scheme, netloc, path, query, fragment, subdomain, domain) are discarded. The `url_length` feature is appended, and the scheme is one-hot encoded into five binary indicators (`is_https`, `is_http`, `is ftp`, `is_none`, `is_other`).
5. **Column reindexing.** The feature vector is reindexed to match the exact column ordering used during training, as recorded in the metadata file.
6. **Prediction.** The LightGBM Booster returns a probability vector $\mathbf{p} \in \mathbb{R}^4$, one entry per class.

7. **Label mapping.** Probabilities are mapped to human-readable class names via the persisted label encoder.

Listing 7: Core inference logic

```
1 def predict_url(url, booster, le, feature_columns, whitelist):
2     X_row = url_to_feature_row(url, feature_columns, whitelist)
3     proba = booster.predict(X_row)[0]
4     labels = le if isinstance(le, np.ndarray) else le.classes_
5     result = {label: round(float(p) * 100, 2)
6               for label, p in zip(labels, proba)}
7     return result
```

15 Feature Engineering

A total of **63 numerical features** are extracted from each URL, organised into six thematically coherent groups. This section provides an exhaustive specification of every feature, its computational definition, and its security rationale. The feature engineering stage constitutes the core intellectual contribution of the system and is the primary determinant of classification performance.

15.1 Structural and Presence Features

This group captures the macro-level composition of the URL by encoding the presence or absence of each structural component and the overall complexity of the URL.

Feature	Type	Description
url_length	int	Total character length of the raw URL string. Malicious URLs—particularly those embedding encoded payloads—tend to exhibit anomalously high lengths.
number_of_part	int	Count of non-empty URL components among {scheme, netloc, path, query, fragment}. Values range from 0 to 5.
has_scheme	bin	Indicates whether a scheme (e.g., http, https, ftp) is explicitly specified.
has_netloc	bin	Indicates the presence of the network location component.
has_path	bin	Indicates the presence of a path component.
has_params	bin	Indicates the presence of RFC 2396 path parameters.

Feature	Type	Description
has_query	bin	Indicates the presence of a query string.
has_fragment	bin	Indicates the presence of a fragment identifier.
has_username	bin	Indicates whether the URL embeds a username in the authority component. This is a well-documented phishing vector, as browsers may display the embedded credentials as if they were part of the domain.
has_password	bin	Indicates the presence of an embedded password in the authority component.
has_port	bin	Indicates whether an explicit port number is specified.
has_subdomain	bin	Indicates the presence of a subdomain (extracted via <code>tldextract</code>).
has_domain	bin	Indicates whether a valid registered domain is present.
has_suffix	bin	Indicates the presence of a recognised top-level domain (TLD) suffix.

Table 18: Group 1: Structural and Presence Features.

Security rationale. Phishing URLs frequently embed credentials within the netloc to deceive users (e.g., `http://paypal.com@evil.com/login`). Malware distribution URLs in the training corpus overwhelmingly specify an explicit scheme, whereas benign URLs harvested from domain lists often omit the scheme.

15.2 Length Features

This group measures the character length of individual URL components. Anomalous long components may indicate obfuscated payloads, excessively deep directory structures, or inflated query strings.

Feature	Type	Description
netloc_length	int	Character length of the network location.
path_length	int	Character length of the path component.
query_length	int	Character length of the query string.

Feature	Type	Description
fragment_length	int	Character length of the fragment identifier.
subdomain_length	int	Character length of the subdomain.
domain_length	int	Character length of the registered domain.

Table 19: Group 2: Length Features.

15.3 Statistical and Entropy Features

Shannon entropy [5] quantifies the degree of randomness within a character sequence. Algorithmically generated domains (AGDs) and machine-generated paths—commonly employed by malware and C2 infrastructure—exhibit characteristically elevated entropy relative to human-authored strings.

The Shannon entropy of a string s of length n with character frequency counts $\{c_1, c_2, \dots, c_k\}$ is computed as:

$$H(s) = \log_2(n) - \frac{1}{n} \sum_{i=1}^k c_i \cdot \log_2(c_i) \quad (1)$$

Feature	Type	Description
url_entropy	float	Shannon entropy computed over the entire URL string.
netloc_entropy	float	Shannon entropy of the network location.
path_entropy	float	Shannon entropy of the path component.
query_entropy	float	Shannon entropy of the query string.
subdomain_entropy	float	Shannon entropy of the subdomain.
domain_entropy	float	Shannon entropy of the registered domain.

Table 20: Group 3: Statistical and Entropy Features.

Listing 8: Shannon entropy implementation

```

1 def Shannon_entropy(data: str) -> float:
2     if not data:
3         return 0
4     char_count = Counter(data)

```

```

5     n = len(data)
6     s = sum(c * log2(c) for c in char_count.values())
7     return log2(n) - s / n

```

15.4 Special Character Features

This group quantifies the occurrence of specific punctuation and non-alphanumeric characters within designated URL components. Certain character patterns serve as strong discriminative signals: for instance, the presence of the @ symbol in the netloc is a well-known URL spoofing technique, and an elevated hyphen count in the subdomain is characteristic of phishing domains (e.g., `secure-login-paypal.attacker.com`).

Feature	Type	Description
hyphen_in_subdomain	int	Count of hyphen characters (-) in the subdomain. Phishing campaigns frequently construct multi-word subdomains mimicking trusted brands.
hyphen_in_domain	int	Count of hyphen characters in the registered domain.
unicode	bin	Indicates whether the domain contains non-ASCII characters (codepoints > 127). This is indicative of Internationalised Domain Name (IDN) homograph attacks.
punycode	bin	Indicates the presence of the xn- prefix, signalling Punycode-encoded internationalised domain names.
at_sign_in_netloc	int	Count of @ characters in the netloc. This character can redirect the browser to a different host (e.g., <code>http://trusted.com@evil.com</code>).
slash_in_path	int	Count of forward slashes (/) in the path, reflecting directory depth.
dot_in_path	int	Count of dots (.) in the path, often corresponding to file extensions.
strange_in_query	int	Count of characters outside the RFC 3986 unreserved and reserved character sets.

Feature	Type	Description
equal_in_query	int	Count of equals signs (=) in the query string, approximating the number of key-value parameter pairs.
ampersand_in_query	int	Count of ampersand characters (&) in the query string.
number_subdomain	int	Number of subdomain levels, computed as the dot count within the subdomain plus one.

Table 21: Group 4: Special Character Features.

Characters deemed “strange” are identified via a regular expression that matches any character not belonging to the RFC 3986 standard character set:

Listing 9: Non-standard character detection pattern (RFC 3986 complement)

```

1 STRANGE_CHAR_PATTERN = re.compile(
2     r"[^a-zA-Z0-9\-\.\_~:/\?#\[\]\@!\$%&'\"\\*\+\,;=%]"
3 )

```

15.5 Lexical and Heuristic Features

This group constitutes the most analytically complex feature category, employing string-similarity algorithms, phonotactic heuristics, and keyword-based threat indicators.

Feature	Type	Description
normalized_ levenshtein_domain	float	Normalised Levenshtein similarity between the host domain and the closest match in the whitelist (see Section 15.5.1). Elevated values indicate potential brand impersonation.
normalized_ levenshtein_subdomain	float	Analogous computation applied to the subdomain component.
random_domain_check	float	Consonant-to-letter ratio in the domain. Algorithmically generated domains (AGDs) tend to exhibit a higher consonant density than natural-language domains.

Feature	Type	Description
random_subdomain_check	float	Consonant ratio computed for the subdomain.
number_ratio_domain	float	Proportion of digit characters within the domain string. Legitimate domains rarely consist predominantly of digits.
number_ratio_subdomain	float	Digit proportion in the subdomain.
repeated_domain_check	float	Ratio of adjacent identical character pairs to total character transitions in the domain.
repeated_path_check	float	Adjacent character repetition ratio in the path.
repeated_url_check	float	Adjacent character repetition ratio across the entire URL.
longest_repeated_chain	int	Length of the longest contiguous run of a single repeated character in the URL.
ip_domain	bin	Indicates whether the domain component is a valid IPv4 or IPv6 address rather than a textual hostname. IP-literal domains are frequently used by malware to bypass domain-based blacklists.
suspicious_key_domain	bin	Indicates whether the domain contains any suspicious keyword (see Table 23).
suspicious_key_subdomain	bin	Suspicious keyword presence in the subdomain.
suspicious_key_path	bin	Suspicious keyword presence in the path.
suspicious_key_query	bin	Suspicious keyword presence in the query string.
shortened	bin	Indicates whether the URL uses a known URL shortening service (e.g., bit.ly, goo.gl, tinyurl.com).

Table 22: Group 5: Lexical and Heuristic Features.

15.5.1 Levenshtein Similarity with the Whitelist

The normalised Levenshtein similarity is the single most discriminative feature for phishing detection. For each input URL, the host domain d is compared against all 10,001 entries in the trusted domain whitelist W . The algorithm employs two optimisation strategies to reduce the computational cost from $O(|W| \cdot |d|^2)$: (i) a *length pre-filter* that skips whitelist entries whose length difference exceeds a dynamic threshold, and (ii) the `score_cutoff` parameter of the RapidFuzz [3] library, which enables early termination of the distance computation.

Algorithm 1 Optimised Levenshtein similarity against the whitelist.

Require: Domain string d ; whitelist $W = \{w_1, w_2, \dots, w_{|W|}\}$

Ensure: Normalised similarity score $s^* \in [0, 1]$

```
1:  $\tau \leftarrow \max(1, \lfloor |d| / 7 \rfloor)$  ▷ Dynamic edit-distance threshold
2:  $s^* \leftarrow 0$ 
3: for each  $w \in W$  do
4:   if  $||d| - |w|| > \tau$  then continue ▷ Length pre-filter
5:   end if
6:    $\delta \leftarrow \text{Levenshtein.distance}(d, w, \text{cutoff}=\tau)$ 
7:   if  $\delta \leq \tau$  then
8:      $s \leftarrow \text{Levenshtein.normalized\_similarity}(d, w)$ 
9:     if  $s > s^*$  then
10:       $s^* \leftarrow s$ ;  $w^* \leftarrow w$ 
11:    end if
12:  end if
13: end for
14: return  $s^*$ 
```

Dynamic threshold rationale. The threshold $\tau = \max(1, \lfloor |d| / 7 \rfloor)$ scales with domain length: short domains (e.g., `fb.com`) permit at most one edit, while longer domains allow proportionally more edits. This prevents false-positive similarity matches on short strings.

15.5.2 Suspicious Keyword Taxonomy

The system maintains a curated lexicon of over 40 keywords, organised by semantic category, that are frequently observed in phishing and social engineering URLs.

Table 23: Suspicious keyword categories and representative terms.

Category	Representative Keywords
Authentication	login, signin, sign-in, log-in, logout
Account	account, myaccount, profile, user
Security	secure, verify, validate, confirm
Financial	banking, payment, checkout, billing, wallet
Urgency	update, urgent, alert, warning, suspend
Support	support, helpdesk, recover, reset, password
Webmail	webmail, outlook, office365, cpanel

15.6 Advanced Cyber Threat Indicators

This final group targets specific threat vectors that have been empirically observed in real-world malware distribution and phishing campaigns.

Feature	Type	Description and Threat Model
has_uuid_path	bin	Detects the presence of a UUID-formatted string (pattern: <code>[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}</code>) within the URL path. UUIDs in paths are a hallmark of C2 server callback channels and automated malware staging.
download_param	bin	Indicates whether the path or query string contains the substrings <code>download</code> or <code>install</code> (applicable only when the respective component exceeds 7 characters in length). This feature targets drive-by download vectors.
free_host	bin	Indicates whether the domain belongs to a known free hosting platform (e.g., <code>vercel.app</code> , <code>netlify.app</code> , <code>github.io</code> , <code>000webhostapp.com</code>). Adversaries frequently exploit free hosting services due to their zero-cost, rapid provisioning, and perceived trustworthiness.

Feature	Type	Description and Threat Model
free_host_download	bin	Conjunction of free_host and download_param. This composite feature provides an exceptionally strong signal for malware distribution via free hosting infrastructure.
suspicious_suffix	bin	Indicates whether the TLD belongs to a curated set of 30 TLDs that are disproportionately associated with abuse (e.g., .tk, .ml, .ga, .cf, .xyz, .top, .click).
is_https	bin	Scheme is https.
is_http	bin	Scheme is http.
is_ftp	bin	Scheme is ftp.
is_none	bin	No scheme is present.
is_other	bin	Scheme is not one of http/https/ftp and is not empty (e.g., data:, javascript:).

Table 24: Group 6: Advanced Cyber Threat Indicators.

Suspicious TLD set. The following 30 TLDs are flagged based on empirical abuse prevalence data:

Listing 10: Set of suspicious top-level domains

```

1 SUSPICIOUS_TLDS = {
2     'tk', 'ml', 'ga', 'cf', 'gq', 'xyz', 'top', 'click',
3     'link', 'win', 'download', 'racing', 'vip', 'club',
4     'online', 'site', 'live', 'fun', 'space', 'pw',
5     'christmas', 'cyou', 'rest', 'icu', 'cfd', 'hair',
6     'makeup', 'monster', 'quest', 'skin',
7 }
```

Free hosting services. The following 15 hosting platforms are monitored:

Listing 11: Set of monitored free hosting domains

```

1 FREE_HOSTING = {
2     'vercel.app', 'netlify.app', 'github.io', 'glitch.me',
3     'replit.dev', 'repl.co', '000webhostapp.com', 'web.app',
4     'firebaseapp.com', 'pages.dev', 'surge.sh', 'onrender.com',
5     'railway.app', 'fly.dev', 'cyclic.app'
6 }
```

15.7 Feature Summary

Table 25: Summary of the six feature groups.

Group	Category	Count
1	Structural & Presence	14
2	Length	6
3	Statistical & Entropy	6
4	Special Characters	11
5	Lexical & Heuristic	16
6	Advanced Threat Indicators	10
Total		63

16 Model Training

16.1 Data Preprocessing

16.1.1 Label Encoding

The four string-valued class labels are transformed into integer codes via `sklearn.preprocessing.LabelEncoder`.

Table 26: Label encoding mapping.

Code	Class
0	benign
1	defacement
2	malware
3	phishing

16.1.2 Feature Matrix Construction

The feature matrix $\mathbf{X} \in \mathbb{R}^{N \times 63}$ is constructed by retaining only the 63 numerical features and discarding all raw textual columns:

Listing 12: Feature matrix construction

```
1 RAW_TEXT_COLUMNS = {"url", "type", "scheme", "netloc", "path",  
2                     "query", "fragment", "subdomain", "domain"}  
3 X = df.drop(columns=RAW_TEXT_COLUMNS)  
4 y = df['type']
```

16.1.3 Deduplication

Prior to partitioning, all exact-duplicate URLs are removed to prevent information leakage through replicated samples:

Listing 13: Deduplication

```

1 url_data = url_data.drop_duplicates(subset='url').reset_index(
    drop=True)

```

16.2 Data Partitioning via Stratified Group K-Fold

A naïve random split (e.g., `train_test_split`) introduces a subtle but significant form of data leakage: URLs sharing the same registered domain (e.g., `example.com/page1` and `example.com/page2`) may be distributed across both the training and test sets. The model can then exploit domain-level memorisation rather than learning generalisable URL-level patterns, resulting in inflated evaluation metrics that do not reflect true out-of-distribution performance.

To address this, we employ `sklearn.model_selection.StratifiedGroupKFold` with $K=10$ folds, using the registered domain as the grouping variable:

Listing 14: Stratified Group K-Fold partitioning

```

1 from sklearn.model_selection import StratifiedGroupKFold
2
3 sgkf = StratifiedGroupKFold(n_splits=10, shuffle=True,
4                             random_state=36)
5 folds = list(sgkf.split(X, y, groups=url_data['domain']))
6
7 test_idx = folds[0][1] # ~10%
8 val_idx = folds[1][1] # ~10%
9 train_idx = np.concatenate([folds[i][1] for i in range(2, 10)])
    # ~80%

```

This partitioning strategy guarantees the following invariants:

- **Stratification.** The class distribution within each fold is approximately proportional to the overall dataset distribution.
- **Group integrity.** All URLs belonging to the same registered domain are confined to a single fold, yielding **zero domain overlap** between the training, validation, and test sets.

Table 27: Data partition statistics.

Partition	Samples	Proportion
Training	529,580	80%
Validation	66,198	10%
Test	66,198	10%
Total	661,976	100%

16.3 LightGBM Hyperparameter Configuration

The classifier is implemented using the LightGBM [1] framework with the Gradient Boosted Decision Trees (GBDT) algorithm. Table 28 presents the complete hyperparameter specification.

Table 28: LightGBM hyperparameter configuration.

Parameter	Value	Rationale
<i>Objective and Task</i>		
objective	multiclass	Multi-class softmax cross-entropy loss.
num_class	4	Four target classes.
boosting_type	gbdt	Gradient Boosted Decision Trees.
class_weight	balanced	Inversely proportional to class frequency.
<i>Capacity Control</i>		
n_estimators	4,000	Maximum number of boosting rounds.
learning_rate	0.025	Conservative step size to promote generalisation.
max_depth	5	Maximum tree depth.
num_leaves	24	Maximum leaf count per tree ($< 2^5 = 32$).
min_data_in_leaf	50	Minimum samples per leaf node.
<i>Stochastic Subsampling</i>		
colsample_bytree	0.75	Feature subsampling rate per tree.
subsample	0.80	Row subsampling rate per boosting round.
subsample_freq	1	Subsampling applied at every iteration.
<i>Regularisation</i>		
reg_alpha	0.1	ℓ_1 (Lasso) regularisation coefficient.
reg_lambda	1.0	ℓ_2 (Ridge) regularisation coefficient.
min_gain_to_split	0.01	Minimum information gain required to perform a split.
<i>Early Stopping</i>		
early_stopping_rounds	100	Training halts if the validation loss fails to improve for 100 consecutive rounds.

16.4 Training Procedure

The model is trained using the scikit-learn-compatible `LGBMClassifier` interface. Two evaluation metrics are monitored concurrently: the primary `multi_logloss` (multi-class cross-entropy) and the secondary `auc_mu` (multiclass area under the ROC curve). Training progress is logged at 100-iteration intervals.

Listing 15: LightGBM training procedure

```
1 model = lgb.LGBMClassifier(**params)
2
3 callbacks = [
4     lgb.early_stopping(stopping_rounds=100, verbose=True),
5     lgb.log_evaluation(100)
6 ]
7
8 model.fit(
9     X_train, y_train,
10    eval_set=[(X_train, y_train), (X_val, y_val)],
11    eval_names=['train', 'val'],
12    callbacks=callbacks,
13    eval_metric=['multi_logloss', 'auc_mu']
14 )
```

Training outcome. Early stopping was triggered at **iteration 738** out of a maximum of 4,000 rounds, indicating that the model achieved convergence well within the allocated capacity. The relatively early termination—at 18.5% of the maximum budget—suggests that the combination of a conservative learning rate ($\eta = 0.025$) and 4,000 estimators provides ample capacity for this classification task.

16.5 Evaluation Results

The trained model is evaluated on the held-out test partition (66,198 samples) with **zero domain overlap** with the training set.

- **Test Accuracy: 93.46%**
- **Evaluation Metrics:** Per-class precision, recall, and F_1 -score via `classification_report`.
- **Visual Diagnostics:** Confusion matrix and train/validation learning curves (`multi_logloss` over iterations).
- **Model Interpretability:** Feature importance (information gain) and SHAP (SHapley Additive exPlanations) value analysis.

17 Model Architecture

17.1 Gradient Boosted Decision Trees

LightGBM [1] implements a highly optimised variant of Gradient Boosted Decision Trees (GBDT), originally proposed by Friedman (2001). The ensemble prediction for a sample $\mathbf{x}_i$ is given by:

$$\hat{y}_i = \sum_{t=1}^T \eta \cdot f_t(\mathbf{x}_i) \quad (2)$$

where T denotes the number of boosting rounds ($T = 738$ after early stopping), f_t is the t -th regression tree, and $\eta = 0.025$ is the learning rate (shrinkage parameter).

Key algorithmic innovations in LightGBM that contribute to its computational efficiency include *Gradient-based One-Side Sampling* (GOSS), which retains samples with large gradients while randomly sampling those with small gradients, and *Exclusive Feature Bundling* (EFB), which bundles mutually exclusive features to reduce the effective feature dimensionality.

17.2 Objective Function

For the four-class classification task, the objective function is the softmax cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \cdot \log(p_{i,c}) \quad (3)$$

where $C = 4$ is the number of classes, $y_{i,c} \in \{0, 1\}$ is the ground-truth indicator, and the class-conditional probability is obtained via the softmax function:

$$p_{i,c} = \frac{\exp(z_{i,c})}{\sum_{j=1}^C \exp(z_{i,j})} \quad (4)$$

with $z_{i,c}$ denoting the raw logit output for sample i and class c .

17.3 Class Balancing

The `class_weight='balanced'` parameter automatically adjusts the sample weight for each class according to:

$$w_c = \frac{N}{C \cdot N_c} \quad (5)$$

where N is the total number of training samples and N_c is the number of samples belonging to class c . This inversely proportional weighting scheme prevents the model from developing a systematic bias toward the majority class.

17.4 Multi-Layer Regularisation Strategy

To mitigate overfitting, the system employs a six-layer regularisation strategy:

1. **Structural constraints:** `max_depth = 5` and `num_leaves = 24` limit the representational capacity of individual trees.
2. **Minimum leaf population:** `min_data_in_leaf = 50` prevents the creation of highly specific leaf nodes.
3. **Feature subsampling:** `colsample_bytree = 0.75` introduces stochasticity by selecting a random subset of features for each tree.
4. **Row subsampling:** `subsample = 0.80` introduces additional randomness through bagging-style sample selection at each iteration.
5. **Explicit regularisation:** ℓ_1 ($\alpha = 0.1$) and ℓ_2 ($\lambda = 1.0$) penalties on the leaf weights.
6. **Minimum split gain:** `min_gain_to_split = 0.01` suppresses splits that yield negligible information gain.

18 Conclusion

This report has presented a complete malicious URL classification system that achieves strong empirical performance through the synergy of principled feature engineering and gradient boosted tree classification. The principal contributions and findings are summarised as follows:

- A test accuracy of **93.46%** is attained on a held-out partition with **zero domain overlap** with the training set, providing a rigorous lower bound on the model’s generalisation capability.
- A comprehensive feature engineering framework comprising **63 handcrafted features** across six thematic groups—spanning structural, statistical, lexical, and threat-specific dimensions—enables effective classification without any network-level or content-level signals.
- The adoption of **Stratified Group K-Fold** partitioning eliminates domain-level data leakage, a pervasive yet frequently overlooked confound in URL classification benchmarks.
- The **LightGBM GBDT** classifier, equipped with a six-layer regularisation strategy, converges at iteration 738 out of 4,000, demonstrating efficient utilisation of model

capacity.

- The inference pipeline operates entirely on the URL string with **sub-millisecond latency**, making it suitable for real-time deployment in network security appliances and browser extensions.

18.1 Limitations and Future Work

Several avenues for improvement merit investigation:

1. **Hyperparameter optimisation.** Systematic tuning via Bayesian optimisation (e.g., Optuna) or random search may yield further performance gains.
2. **Ensemble methods.** Combining LightGBM with complementary classifiers such as XGBoost or Random Forest through weighted averaging or stacking may improve robustness.
3. **Feature augmentation.** Incorporating WHOIS registration age, SSL certificate attributes, and passive DNS metadata could enhance discriminative power, albeit at the cost of increased inference latency.
4. **Dynamic thresholding.** Adapting the classification decision boundary based on contextual signals (e.g., geographic origin, time of day) could improve precision in deployment-specific scenarios.
5. **Deployment.** Integration into a REST API or browser extension would enable real-time URL screening for end users.

References

- [1] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. *Advances in Neural Information Processing Systems* 30 (NeurIPS), 3149–3157.
- [2] Kurkowski, J. *tldextract* — *Accurately separate the TLD from the registered domain and subdomains of a URL*. <https://github.com/john-googling/tldextract>
- [3] Bachmann, M. *RapidFuzz* — *Rapid fuzzy string matching in Python and C++*. <https://github.com/maxbachmann/RapidFuzz>
- [4] abuse.ch. *URLhaus* — *Malware URL exchange*. <https://urlhaus.abuse.ch/>
- [5] Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3), 379–423.
- [6] Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707–710.
- [7] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [8] Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189–1232.
- [9] Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems* 30 (NeurIPS), 4765–4774.